

Attention is all you need!



Sommaire

- Introduction
- Détails du modèle
 - Encoder
 - Decoder
- Résultats
- Conclusion



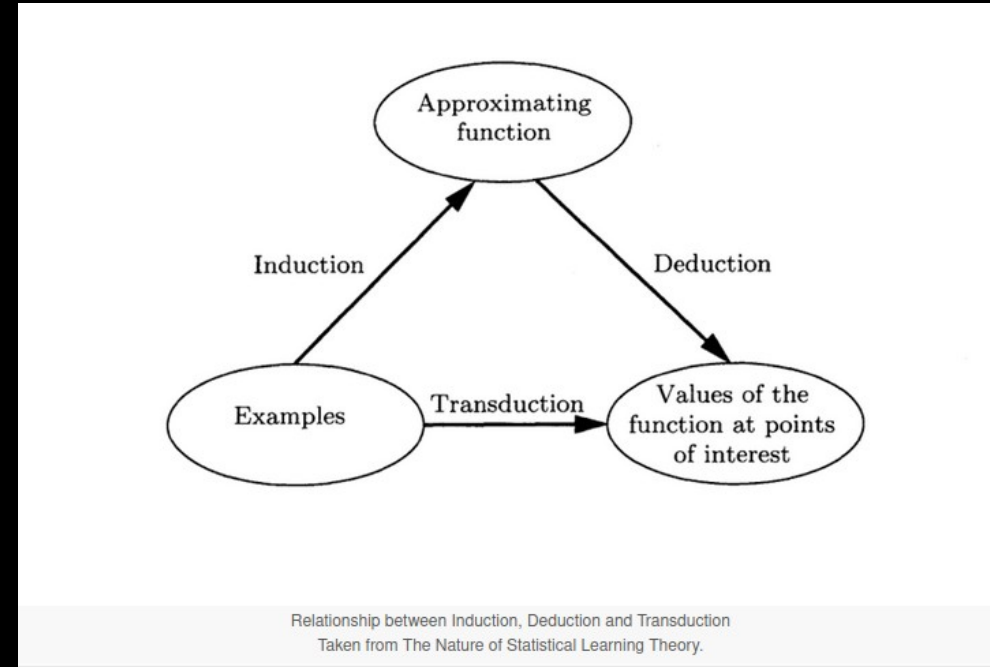
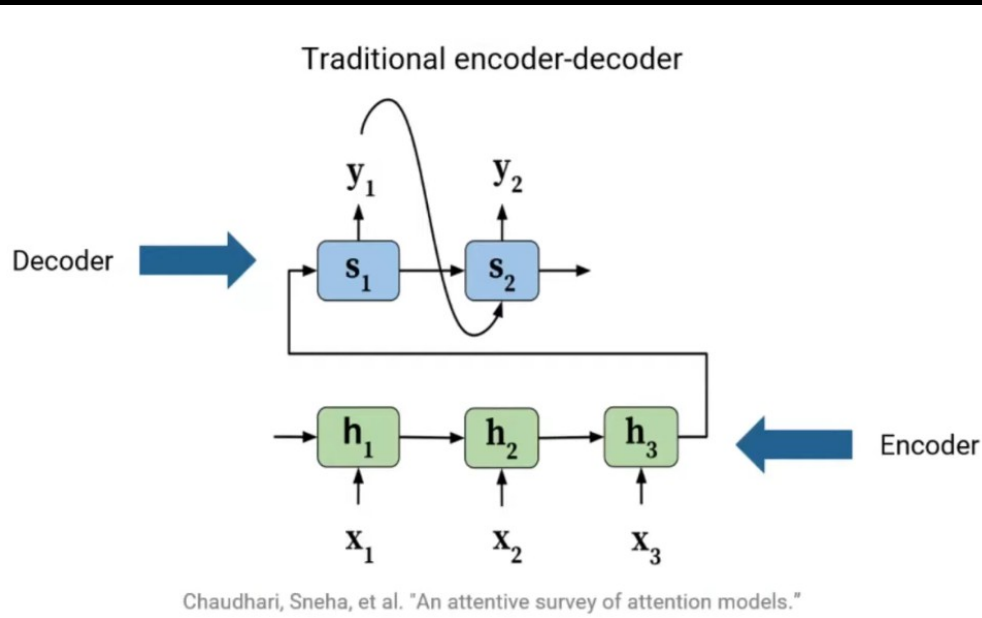
Introduction : c'est quoi?



Introduction

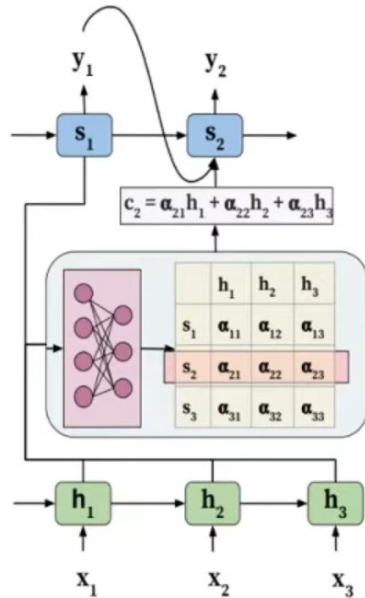
Transduction et modélisation de séquences :

- Modélisation du langage
- Traduction
- RNNs, LSTM, gated RNNs



Introduction

Encoder-decoder model with Attention



Chaudhari, Sneha, et al. "An attentive survey of attention models."

- 2014 : Neural Machine Translation by Jointly Learning to Align and Translate" by Bahdanau et al.
- Affinités entre éléments distants
- Mais RNNs trop lents (GPU...)



Introduction



Cornell University

arXiv > cs > arXiv:1706.03762

Computer Science > Computation and Language

[Submitted on 12 Jun 2017 (v1), last revised 6 Dec 2017 (this version, v5)]

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

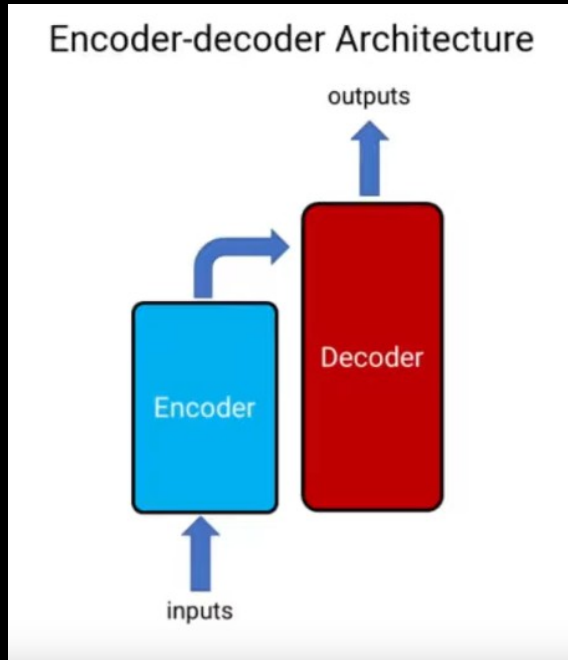
2017 : Introduction d'une nouvelle architecture : les transformers.
Pas de traitement séquentiel --> //



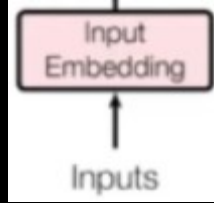
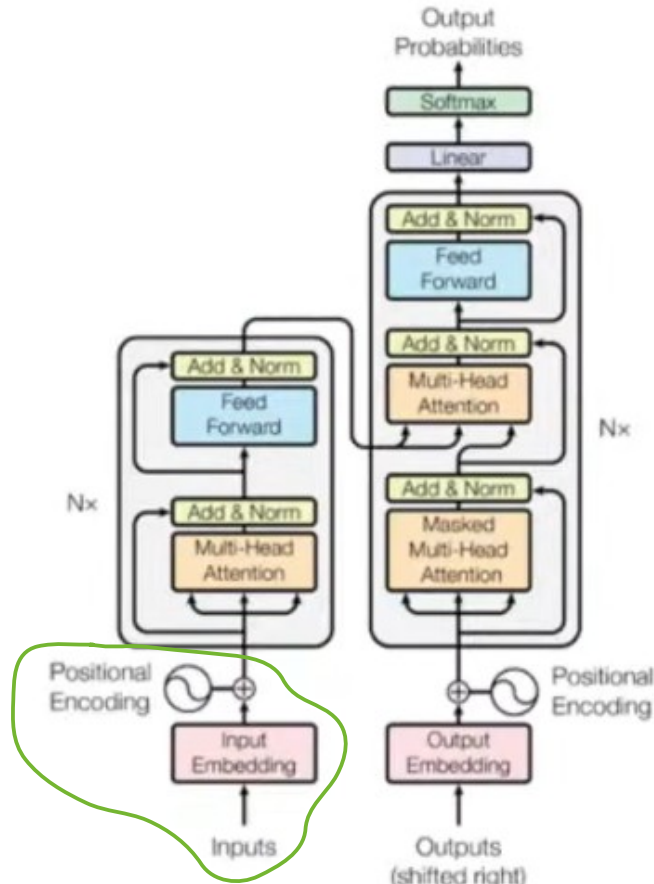
Détails du modèle Encoder (& Decoder)



Détails de l'encoder



Transformer Architecture



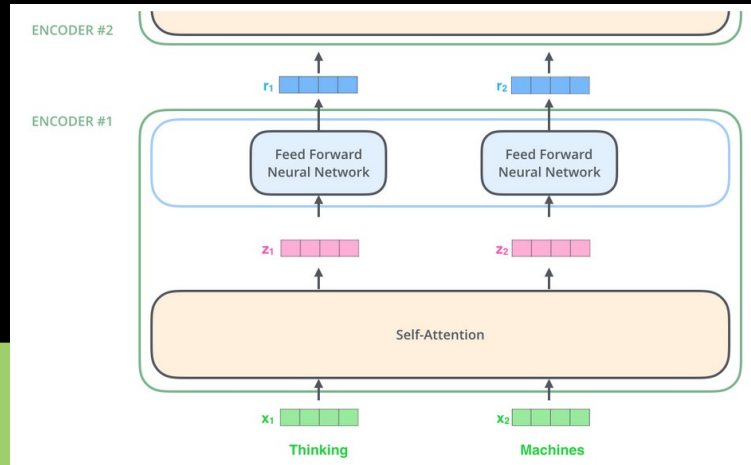
Inputs = séquence de mots

Comment encoder ces mots?

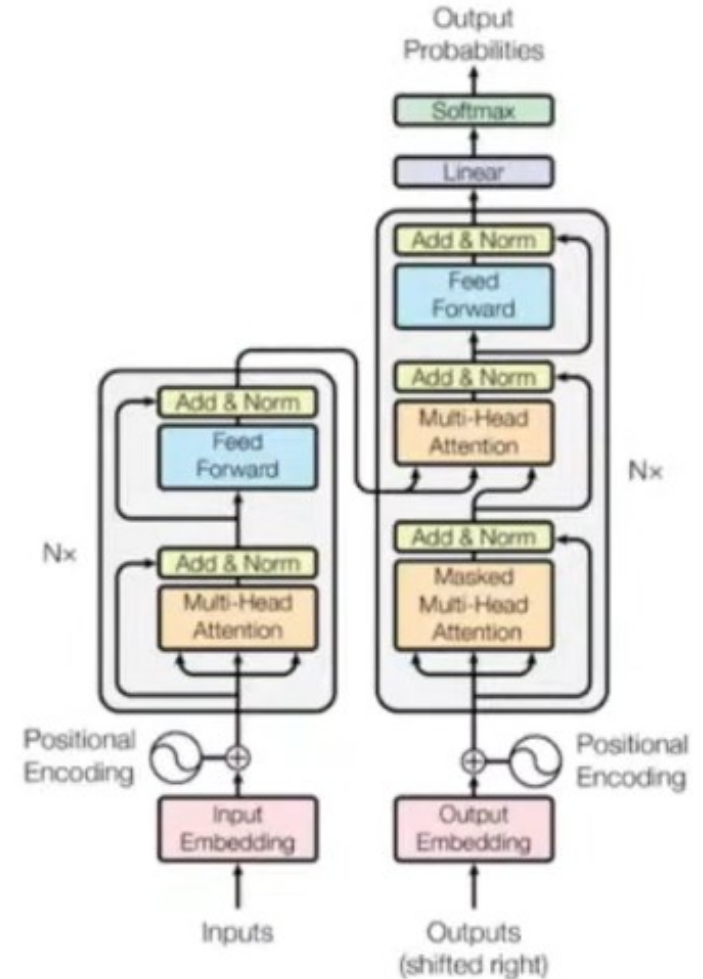


Encoder

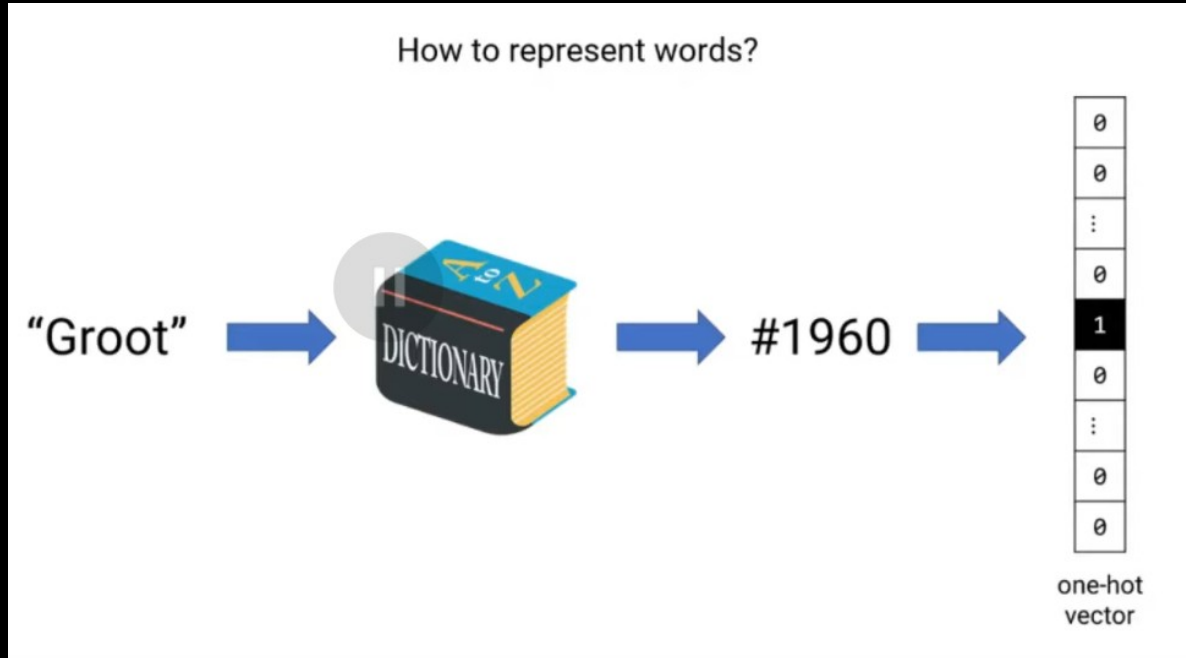
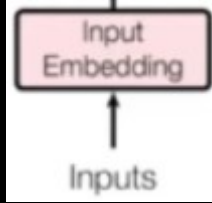
1. Inputs embedding
2. Positional encoding
3. Multi-head self-attention
4. Add & Norm
5. Feed forward
6. Add & Norm



Transformer Architecture



Preprocessing : inputs embedding



Première idée : one-hot vector

-- > One hot encoder

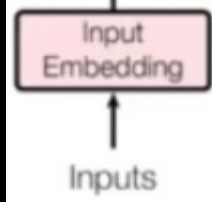
One-Hot Encoding

datagy.io

Island	Biscoe	Dream	Torgensen
Biscoe	1	0	0
Torgensen	0	0	1
Dream	0	1	0



Preprocessing : inputs embedding



```
# Loading a Sample DataFrame
import pandas as pd

df = pd.DataFrame({
    'Name': ['Joan', 'Matt', 'Jeff', 'Melissa', 'Devi'],
    'Gender': ['Female', 'Male', 'Male', 'Female', 'Female'],
    'House Type': ['Apartment', 'Detached', 'Apartment', None,
'Semi-Detached']
})
```

```
print(df)
```

```
# Returns:
```

#	Name	Gender	House Type
# 0	Joan	Female	Apartment
# 1	Matt	Male	Detached
# 2	Jeff	Male	Apartment
# 3	Melissa	Female	None
# 4	Devi	Female	Semi-Detached

```
# Understanding the Pandas get_dummies function
```

```
import pandas as pd
```

```
pd.get_dummies(
```

```
    data,
```

```
    prefix=None,
```

```
    prefix_sep='_',
```

```
    dummy_na=False,
```

```
    columns=None,
```

`data=` represents the data from which to get the dummy indicators (either array-like, Pandas Series, or Pandas DataFrame)

`prefix=` represents the string to append to DataFrame column names

`prefix_sep=` represents what delimiter to use

`dummy_na=` represents whether to add a column or not for missing values

`columns=` represents the names of the columns to be encoded

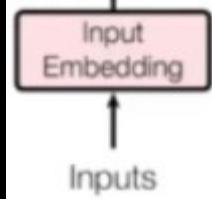
`sparse=` represents whether the data should be a sparse array or a regular [NumPy](#)

array

`drop_first=` represents whether to drop the first level or not

`dtype=` represents the data type for new columns

Preprocessing : inputs embedding



```
# One-Hot Encoding a Single DataFrame Series
```

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['Joan', 'Matt', 'Jeff', 'Melissa', 'Devi'],
    'Gender': ['Female', 'Male', 'Male', 'Female', 'Female'],
    'House Type': ['Apartment', 'Detached', 'Apartment', None,
'Semi-Detached']
})
```

```
print(pd.get_dummies(df['Gender']))
```

```
# Returns:
```

#	Female	Male
# 0	1	0
# 1	0	1
# 2	0	1
# 3	1	0
# 4	1	0

```
# One-Hot Encoding and Returning a DataFrame
```

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['Joan', 'Matt', 'Jeff', 'Melissa', 'Devi'],
    'Gender': ['Female', 'Male', 'Male', 'Female', 'Female'],
    'House Type': ['Apartment', 'Detached', 'Apartment', None,
'Semi-Detached']
})
```

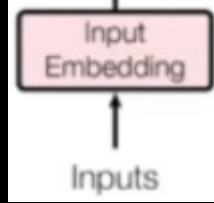
```
ohe = pd.get_dummies(data=df, columns=['Gender'])
```

```
print(ohe)
```

```
# Returns:
```

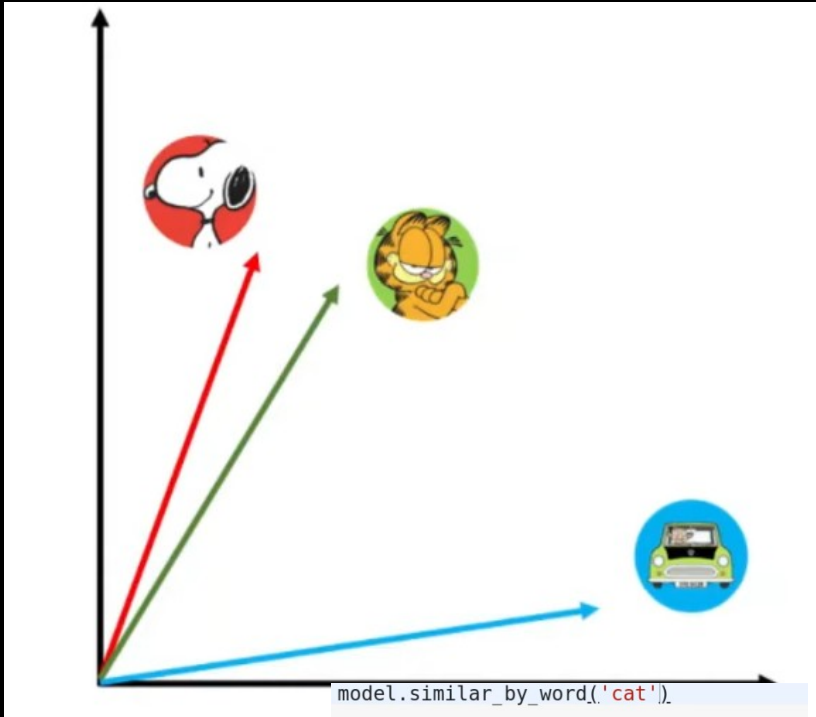
#	Name	House Type	Gender_Female	Gender_Male
# 0	Joan	Apartment	1	0
# 1	Matt	Detached	0	1
# 2	Jeff	Apartment	0	1
# 3	Melissa	None	1	0
# 4	Devi	Semi-Detached	1	0

Preprocessing : inputs embedding



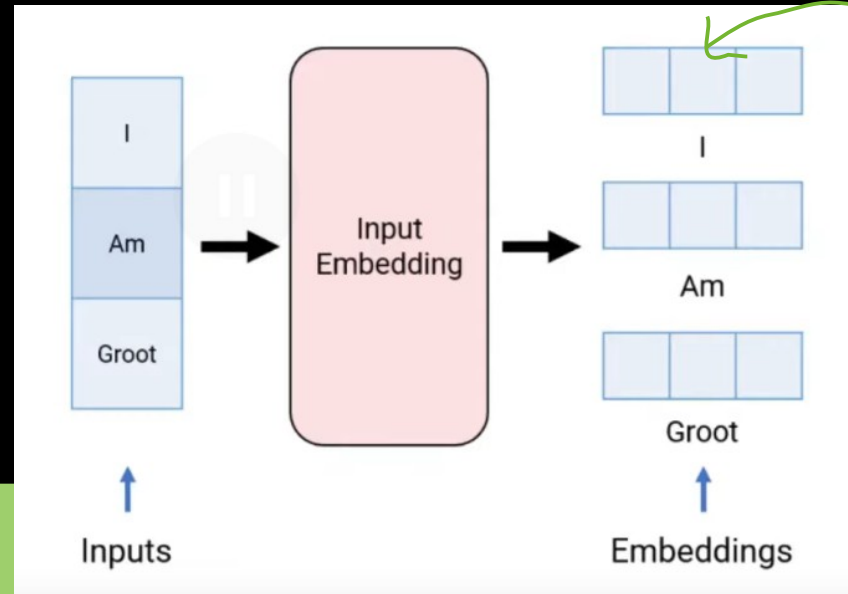
Meilleure idée : word embeddings

Mots similaires $\leftrightarrow$
représentations similaires



```
model.similar_by_word('cat').
```

```
/usr/local/lib/python3.6/dist-packages\nif np.issubdtype(vec.dtype, np.int)\n[('cats', 0.8099379539489746),\n ('dog', 0.7609456777572632),\n ('kitten', 0.7464985251426697),\n ('feline', 0.7326233983039856),\n ('H...', 0.7150502067011014)]
```



512

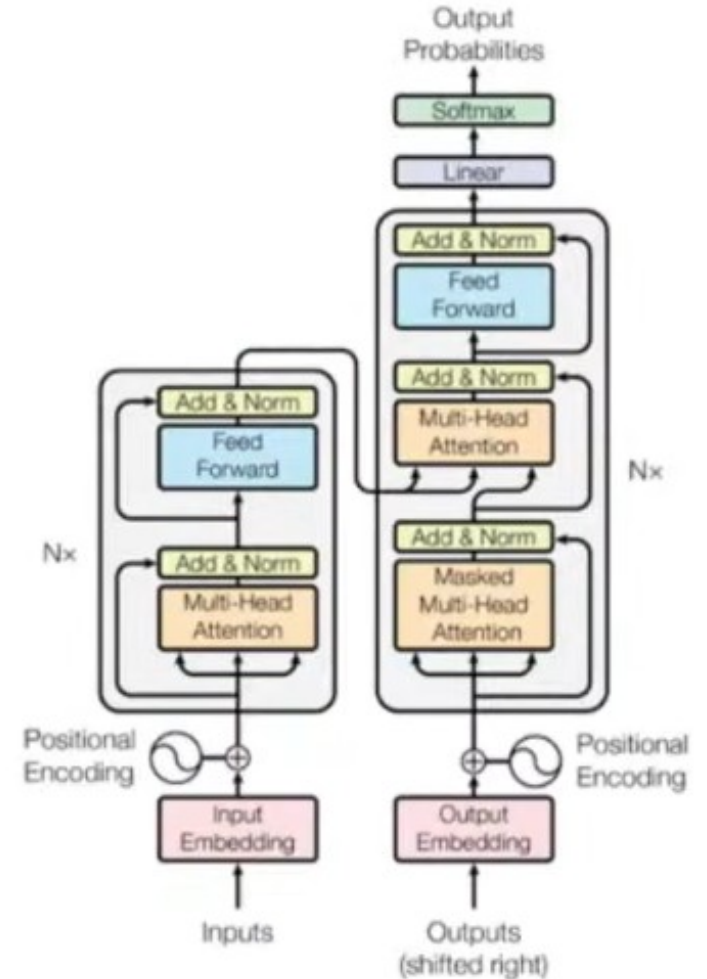


Encoder

1. Inputs embedding
- 2. Positional encoding**
3. Multi-head self-attention
4. Add & norm
5. Feed forward
6. Add & norm

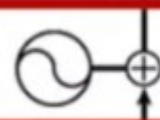


Transformer Architecture



Preprocessing : positional encoding

Positional
Encoding



CHECKLIST

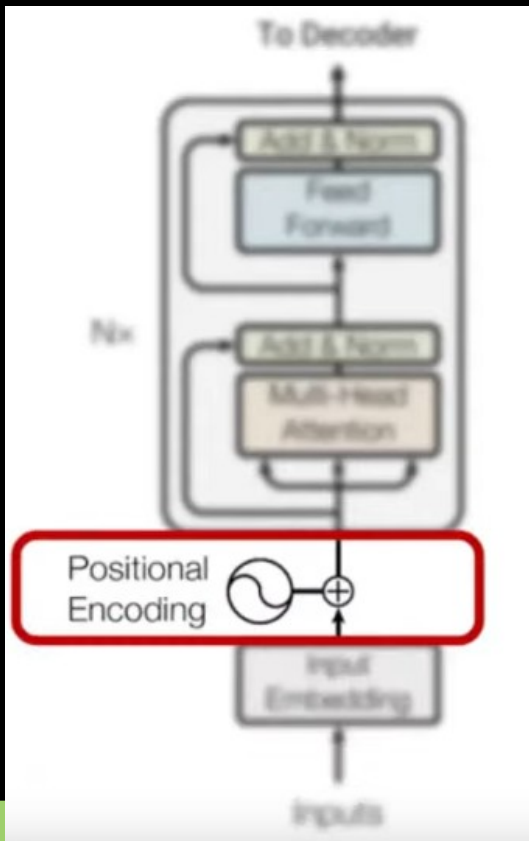
- Unique encoding for each time-step
- Consistent distance between any two time-steps
- should generalize to longer sentences
- deterministic

Positional encoding = rajouter de l'info sur les positions relatives des mots

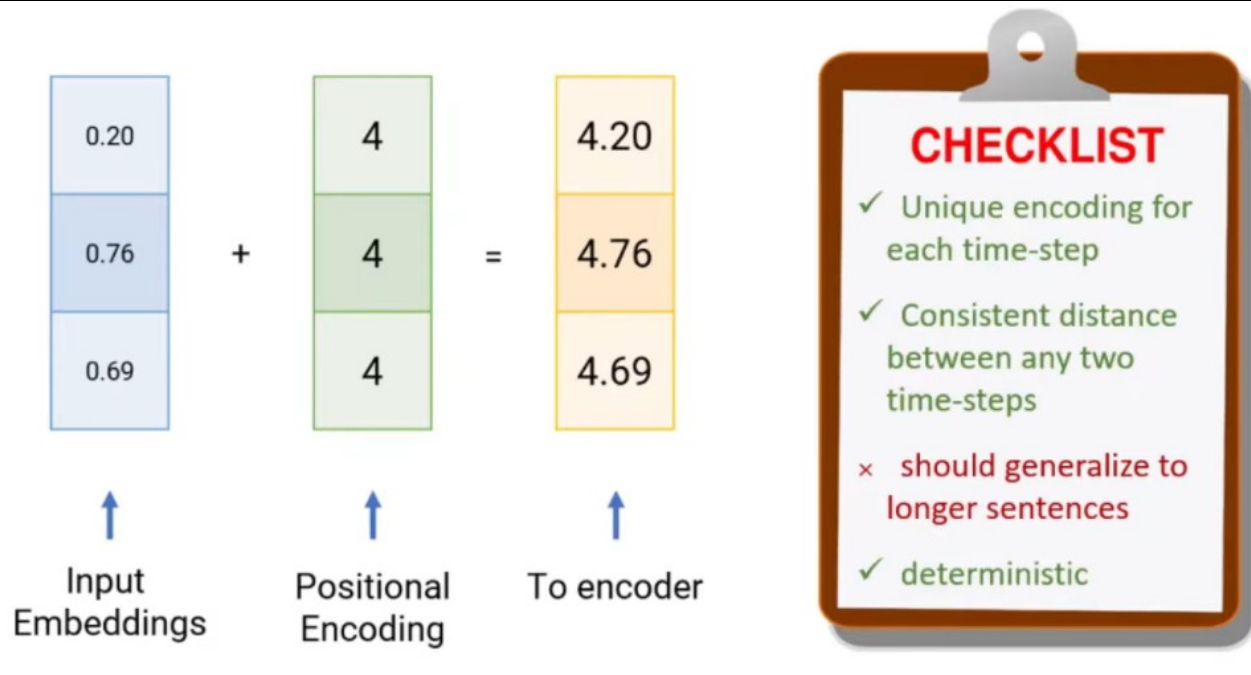
-- > Faire la somme d'un nouveau vecteur (positions relatives) et des vecteurs précédents (mots)

The Avengers defeated Thanos

Thanos defeated The Avengers



Preprocessing : positional encoding

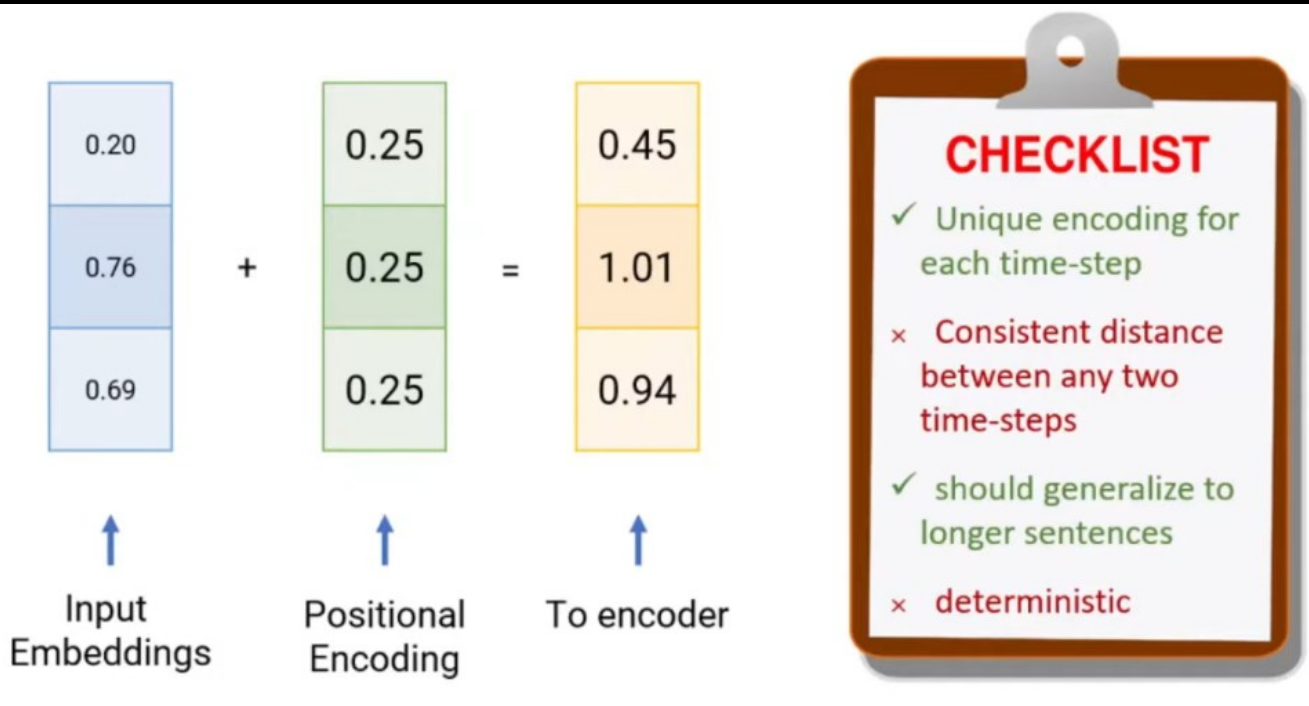


Rajouter 1 à la somme des vecteurs précédents pour chaque nouveau mot...

>>



Preprocessing : positional encoding



CHECKLIST

- ✓ Unique encoding for each time-step
- × Consistent distance between any two time-steps
- ✓ should generalize to longer sentences
- × deterministic

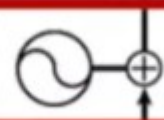
Rajouter un nombre entre 0 et 1

!deterministe <- plusieurs mots / delta T

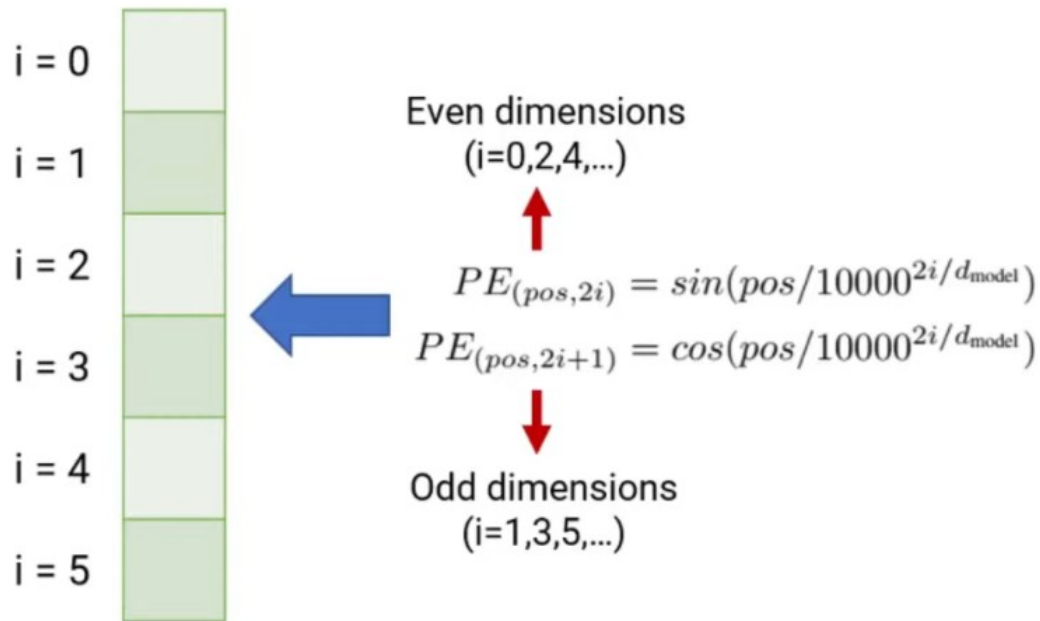


Preprocessing : positional encoding

Positional
Encoding



How to create the Positional Encoding (Transformer style)



Positional
Encoding

Vecteur de 512 lignes

$d_{\text{model}}=512$

$$\vec{p}_t = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_{d \times 1} \quad \left(\omega_k = \frac{1}{10000^{\frac{2k}{d}}} \right)$$

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

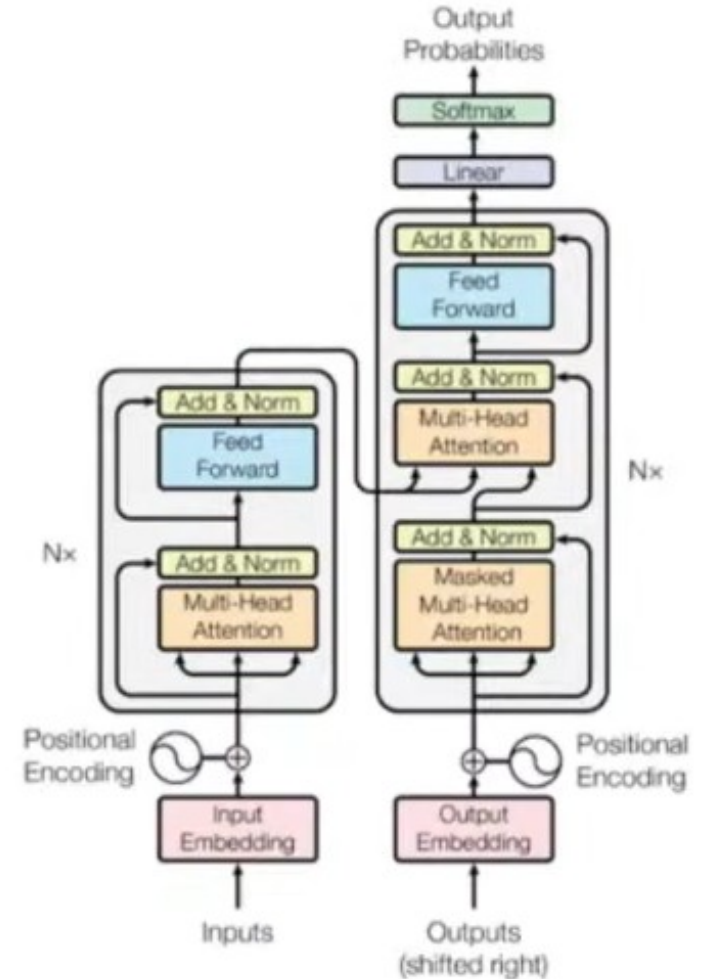
PE permet de donner des données sur l'ordre des inputs



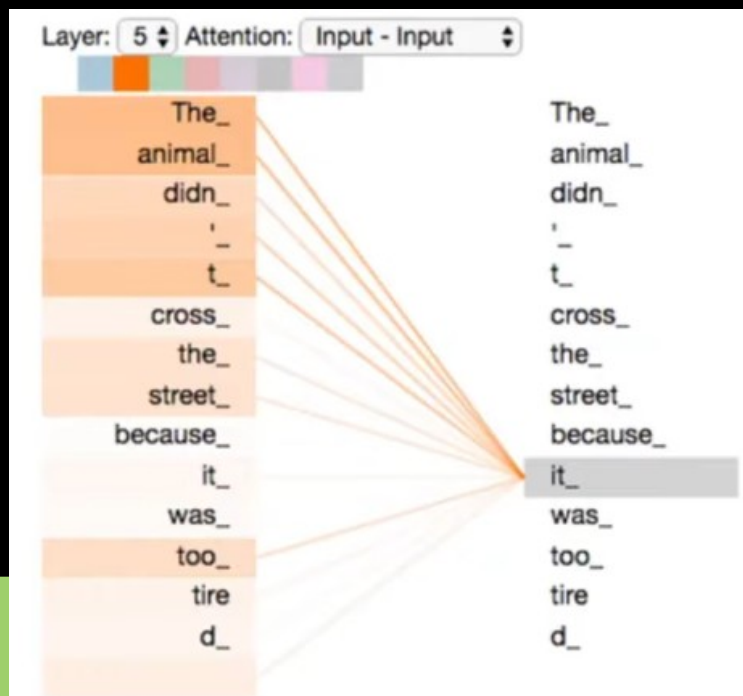
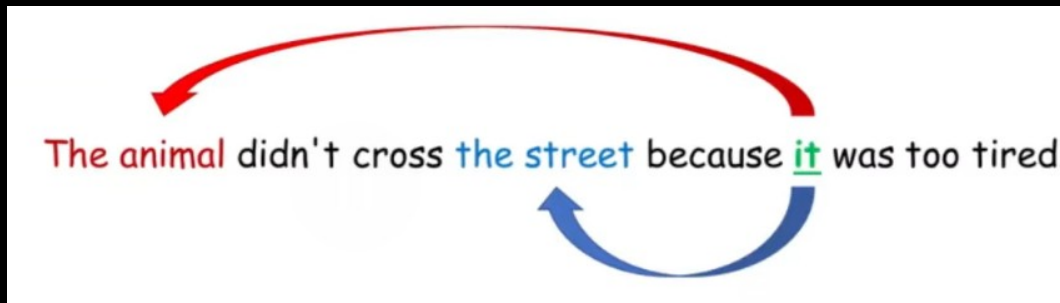
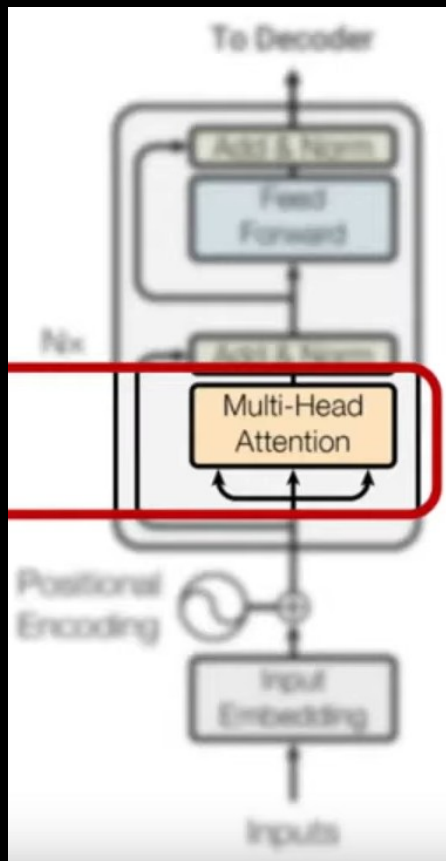
Encoder

1. Inputs embedding
2. Positional encodding
- 3. Multi-head self-attention**
4. Add & Norm
5. Feed forward
6. Add & Norm

Transformer Architecture



(Multi-head) self-attention



(Multi-head) self-attention

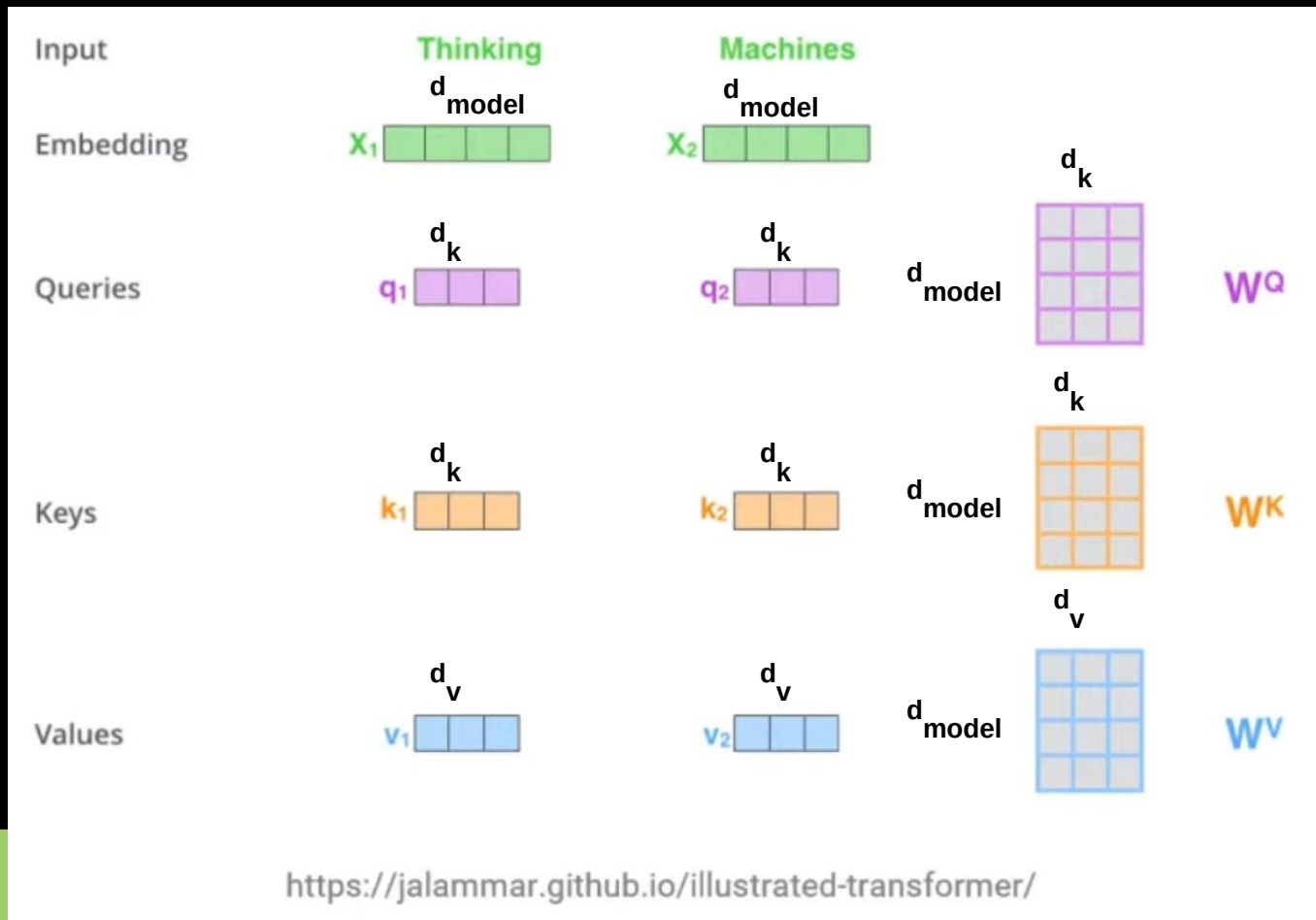
Légende :

$d_k=64$

$q_1 = X_1 * W_q$

$k_1 = X_1 * W_k$

$v_1 = X_1 * W_v$



Query = ce que je cherche, mot/position courante qui porte attention id est celui qui interroge les autres mots

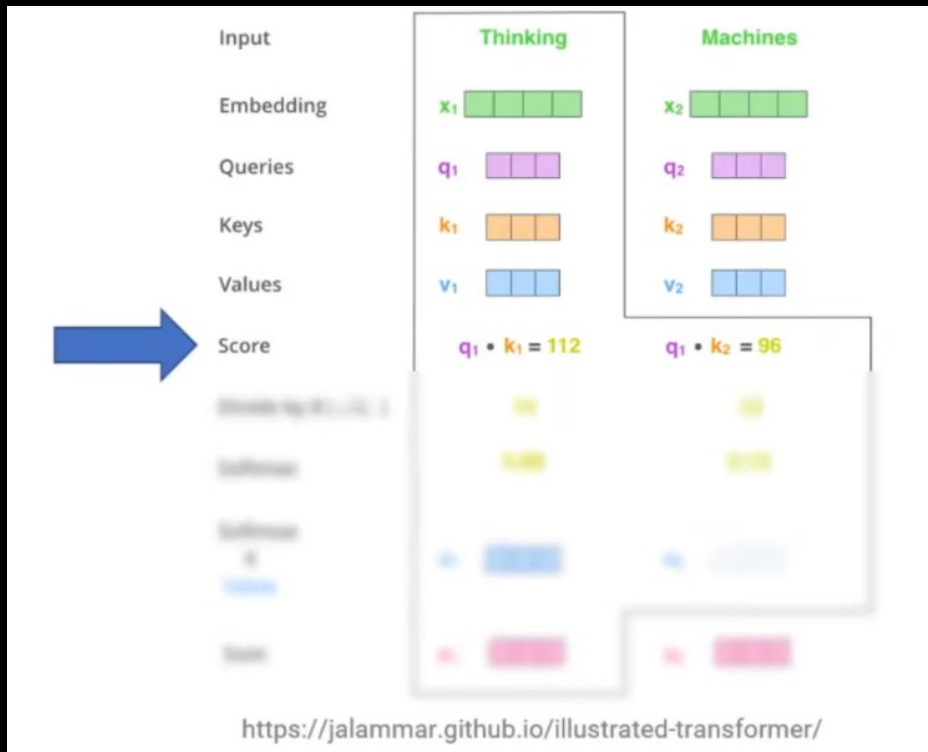
Key = ce que je contiens, encode le mot sur lequel l'attention est portée

Value = ce que je donne si tu me veux



(Multi-head) self-attention

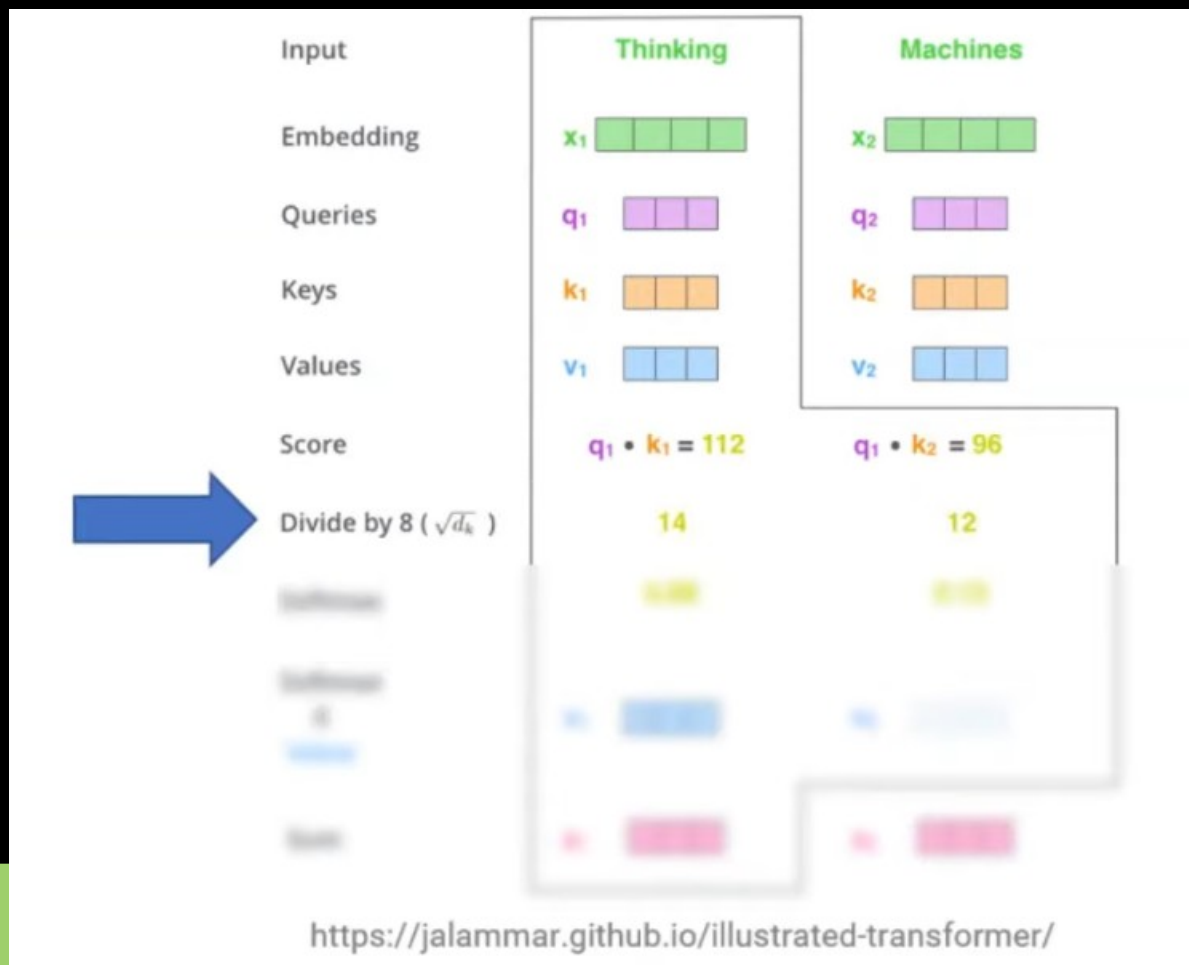
Score >>
si key et
query sont
alignés...
Produit
scalaire
donne un
score
d'attention
élevé



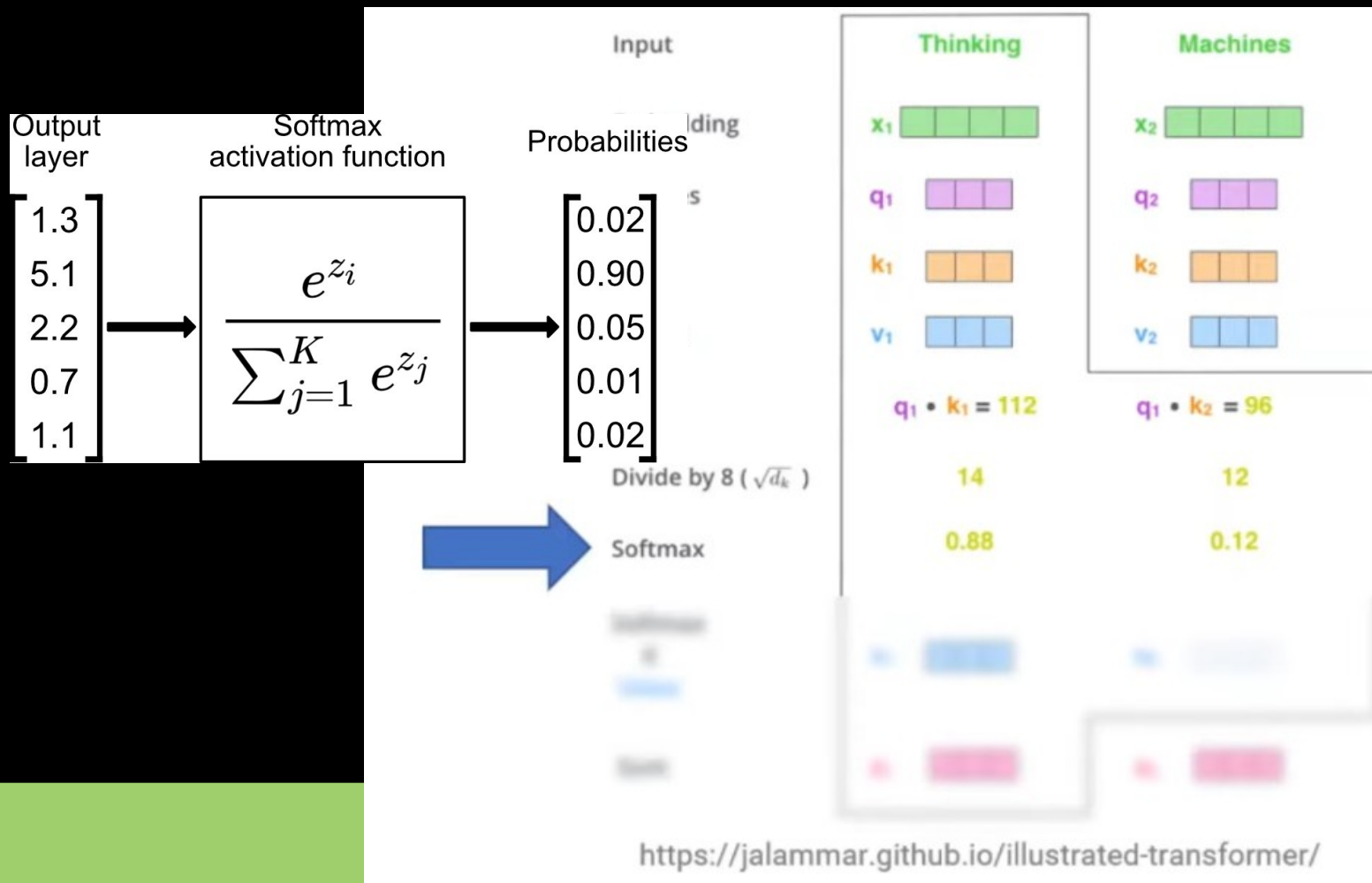
Pour calculer l'auto-attention du premier mot "Thinking", on doit avoir un score pour chaque mot de l'entrée par rapport à ce mot



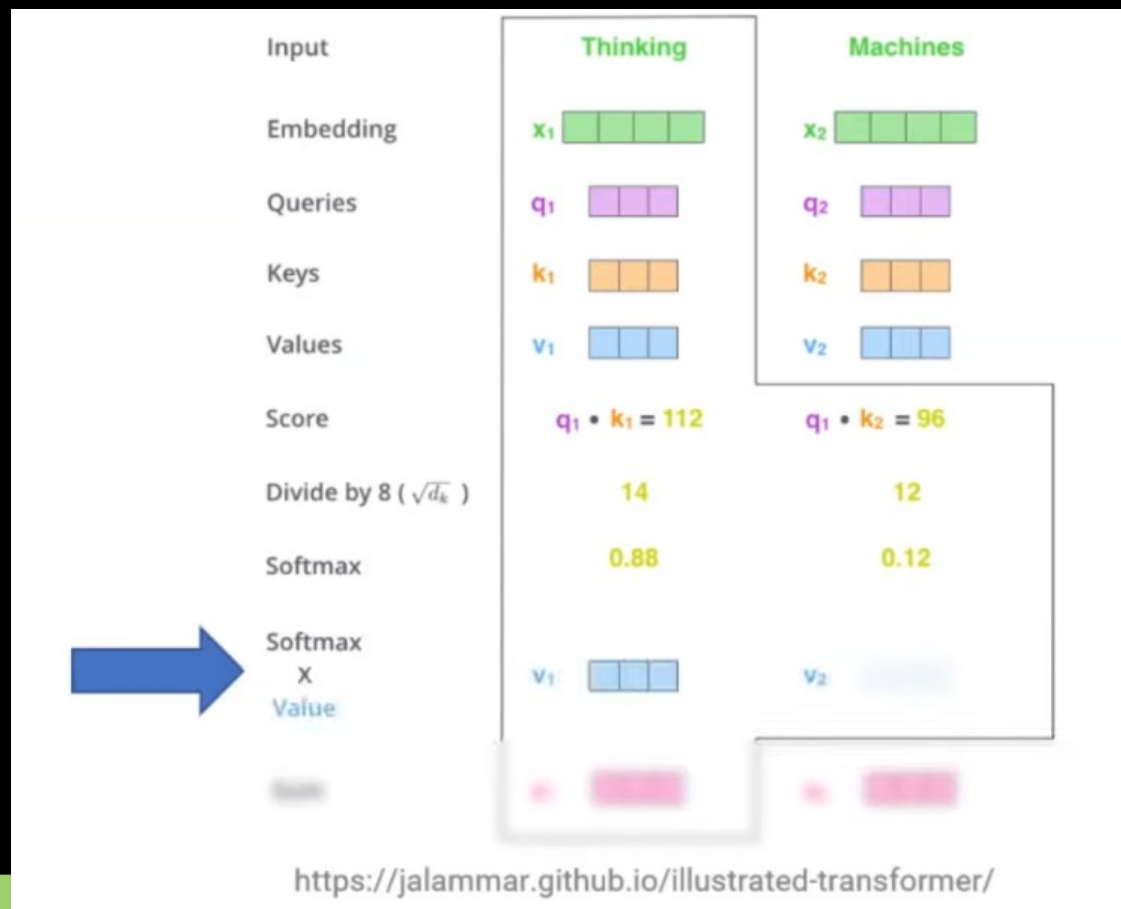
(Multi-head) self-attention



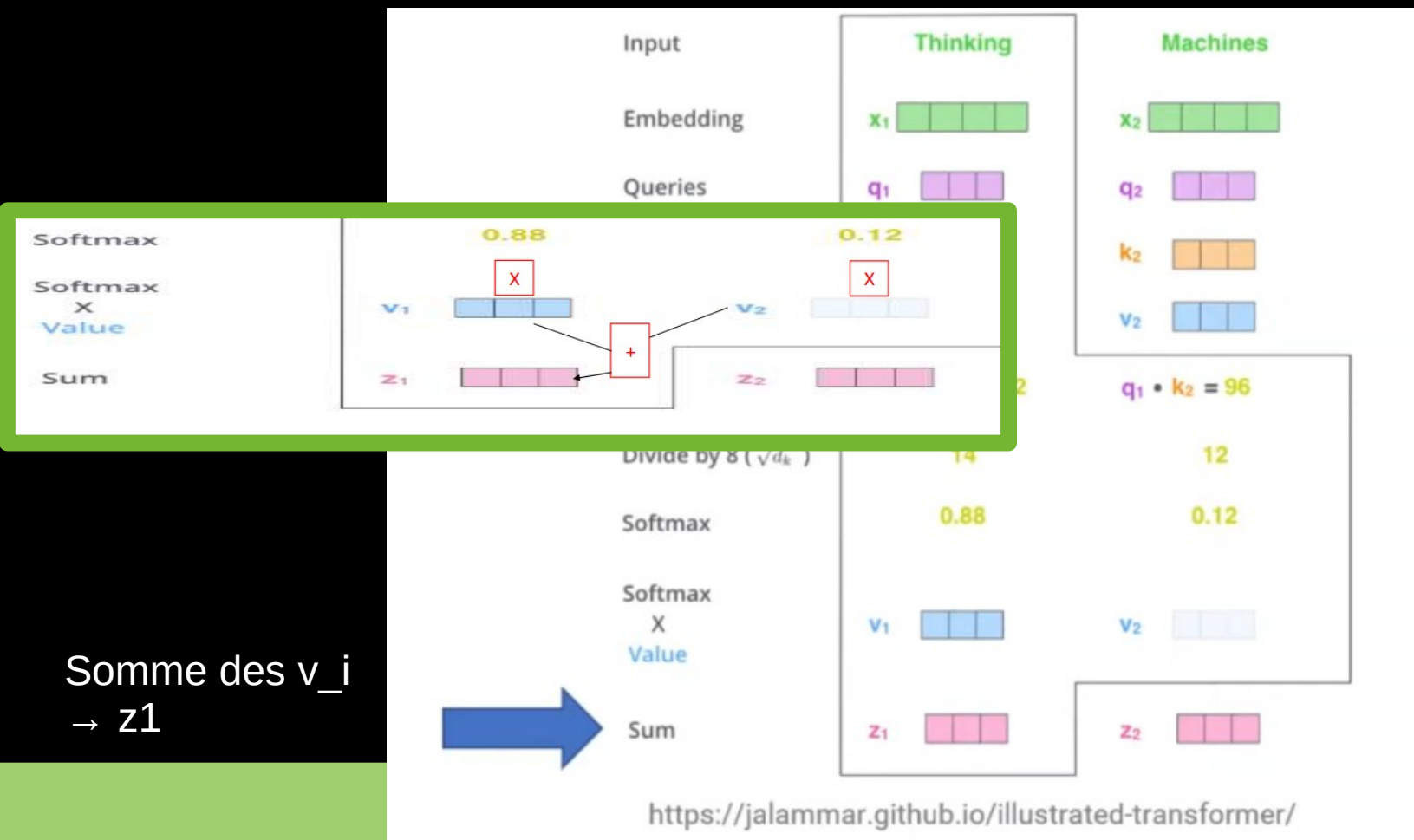
(Multi-head) self-attention



(Multi-head) self-attention



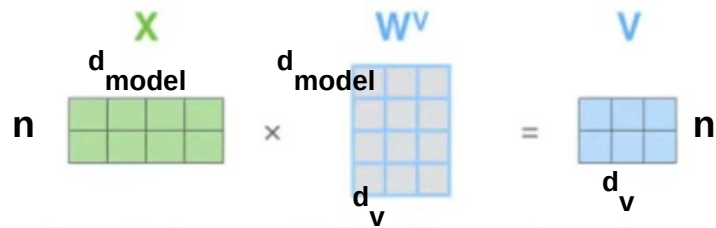
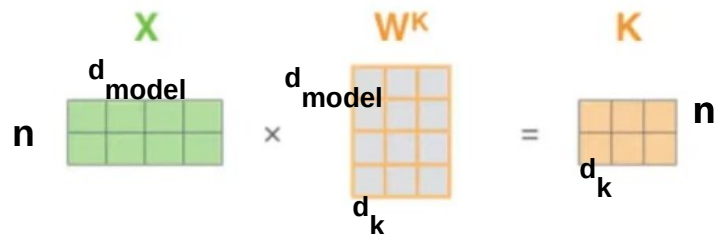
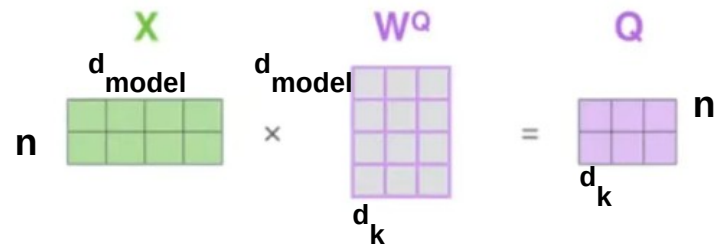
(Multi-head) self-attention



Somme des v_i
→ z_1



(Multi-head) self-attention



<https://jalammar.github.io/illustrated-transformer/>

$$\text{softmax} \left(\frac{n \times d_k}{\sqrt{d_k}} \right) n \times d_v = n \times d_v$$

The diagram shows the calculation of the attention weights. The query Q (size $n \times d_k$) is multiplied by the transpose of the key K^T (size $n \times d_k$). The result is divided by $\sqrt{d_k}$. The softmax function is applied to the result to produce the attention weights (size $n \times n$). These weights are then multiplied by the value V (size $n \times d_v$) to produce the final output Z (size $n \times d_v$).

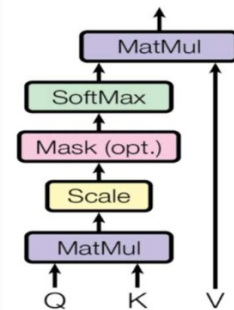
<https://jalammar.github.io/illustrated-transformer/>

Mais du coup pourquoi
multi head...?

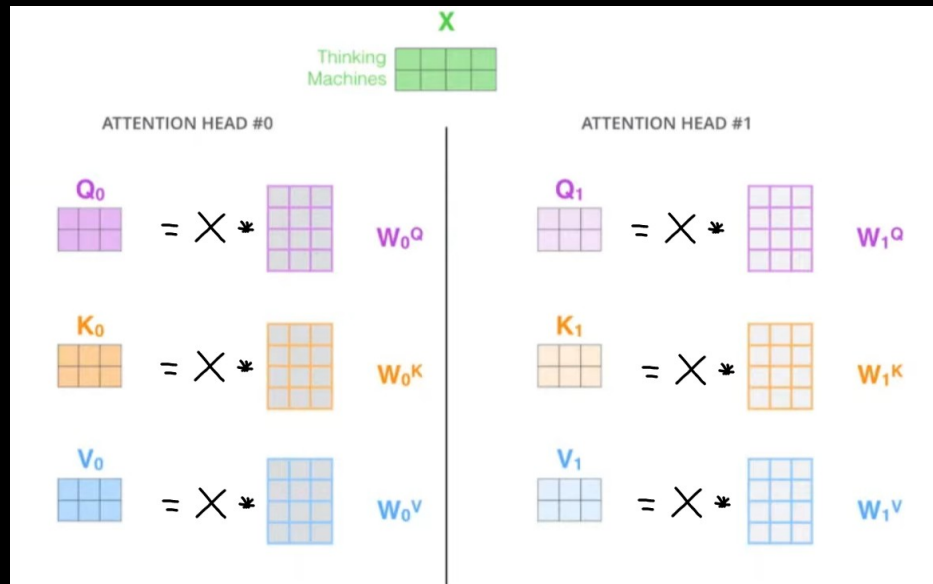
Fonction d'auto-attention: *Scaled dot product*

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

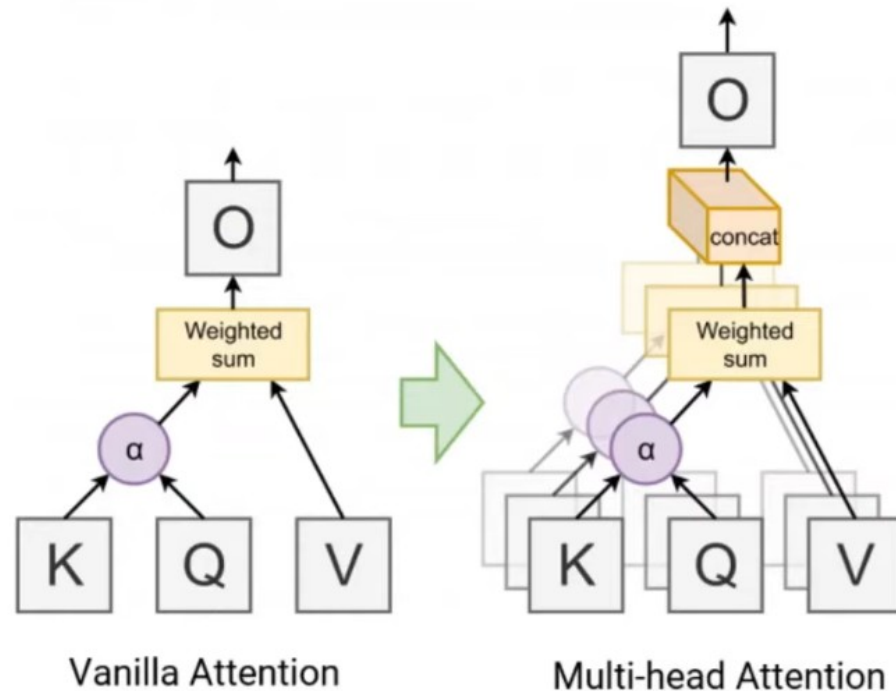
d_k = la dimension des vecteurs de clé
Q, K, V : Vecteurs de requête, clé et valeur



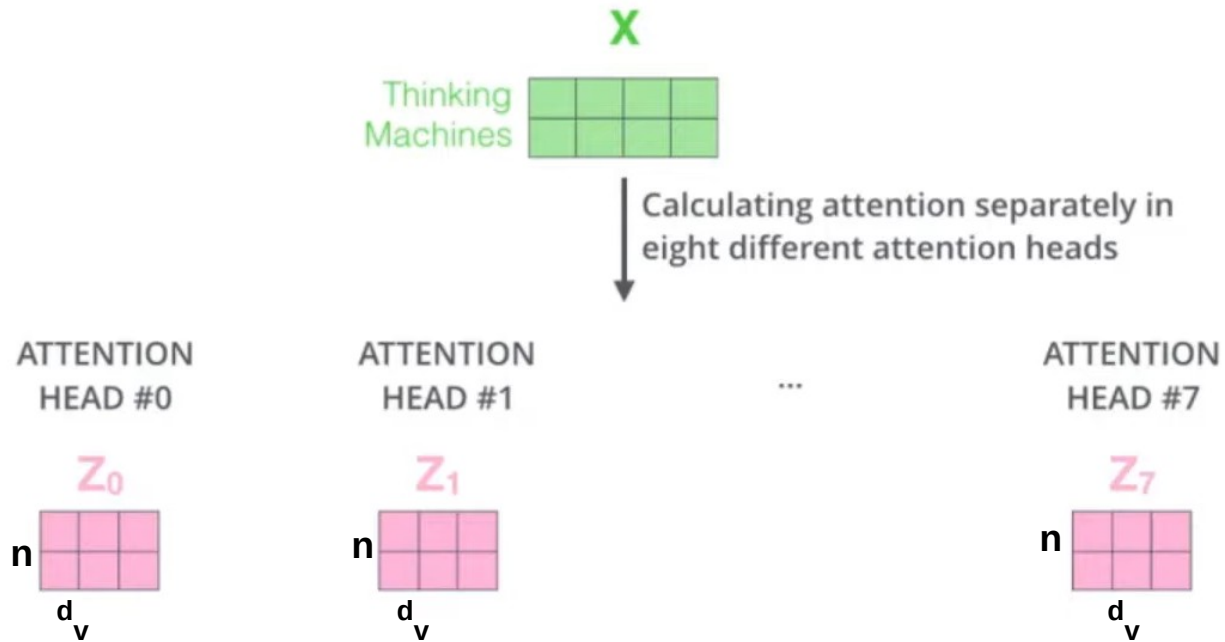
Multi-head self-attention



Exemple avec deux heads
→ augmente capacité
expressive du modèle



Multi-head self-attention

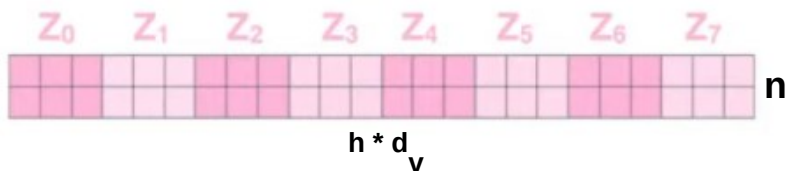


<https://jalammar.github.io/illustrated-transformer/>



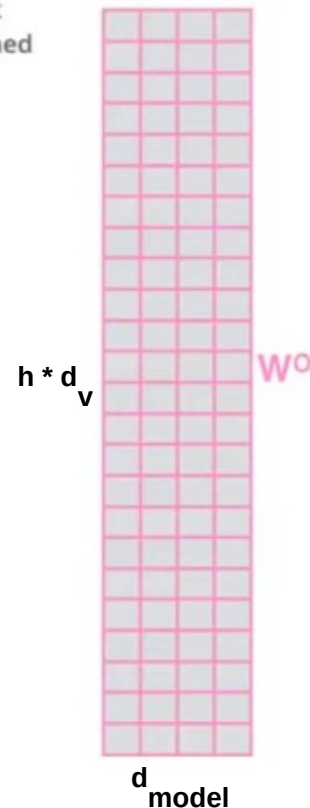
Multi-head self-attention

1) Concatenate all the attention heads

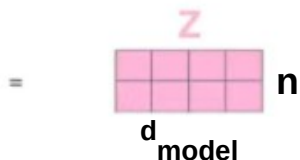


2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



<https://jalammar.github.io/illustrated-transformer/>

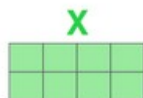
Notation :
 $h=8$

Multi-head self-attention

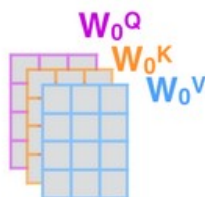
1) This is our
input sentence*

Thinking
Machines

2) We embed
each word*



3) Split into 8 heads.
We multiply X or
 R with weight matrices



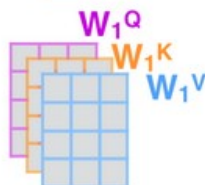
4) Calculate attention
using the resulting
 $Q/K/V$ matrices



5) Concatenate the resulting Z matrices,
then multiply with weight matrix W^O to
produce the output of the layer



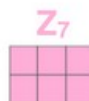
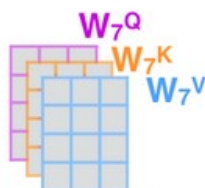
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



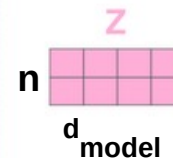
...

...

...



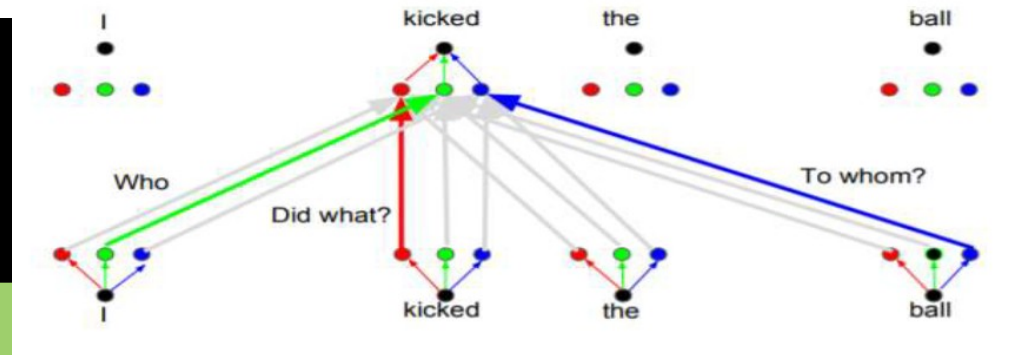
W^O



Multi-head self-attention

Attention multi-têtes (*Multi-head Attention*)

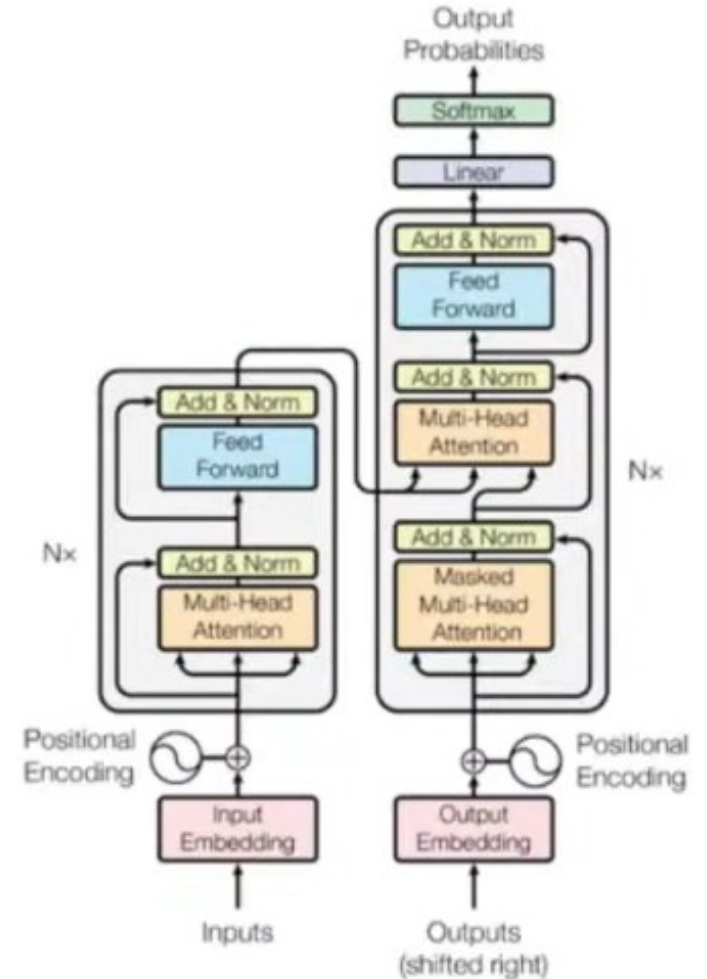
- Le **modèle d'attention multi-têtes** élargit la capacité du modèle à se concentrer sur différentes positions
- Il donne à la couche d'attention plusieurs «sous-espaces de représentation»
- Avec une attention multiple, nous avons plusieurs ensembles de matrices de pondération *Query / Key / Value*
- Le Transformateur utilise **huit têtes d'attention**, nous avons donc huit ensembles pour chaque encodeur.



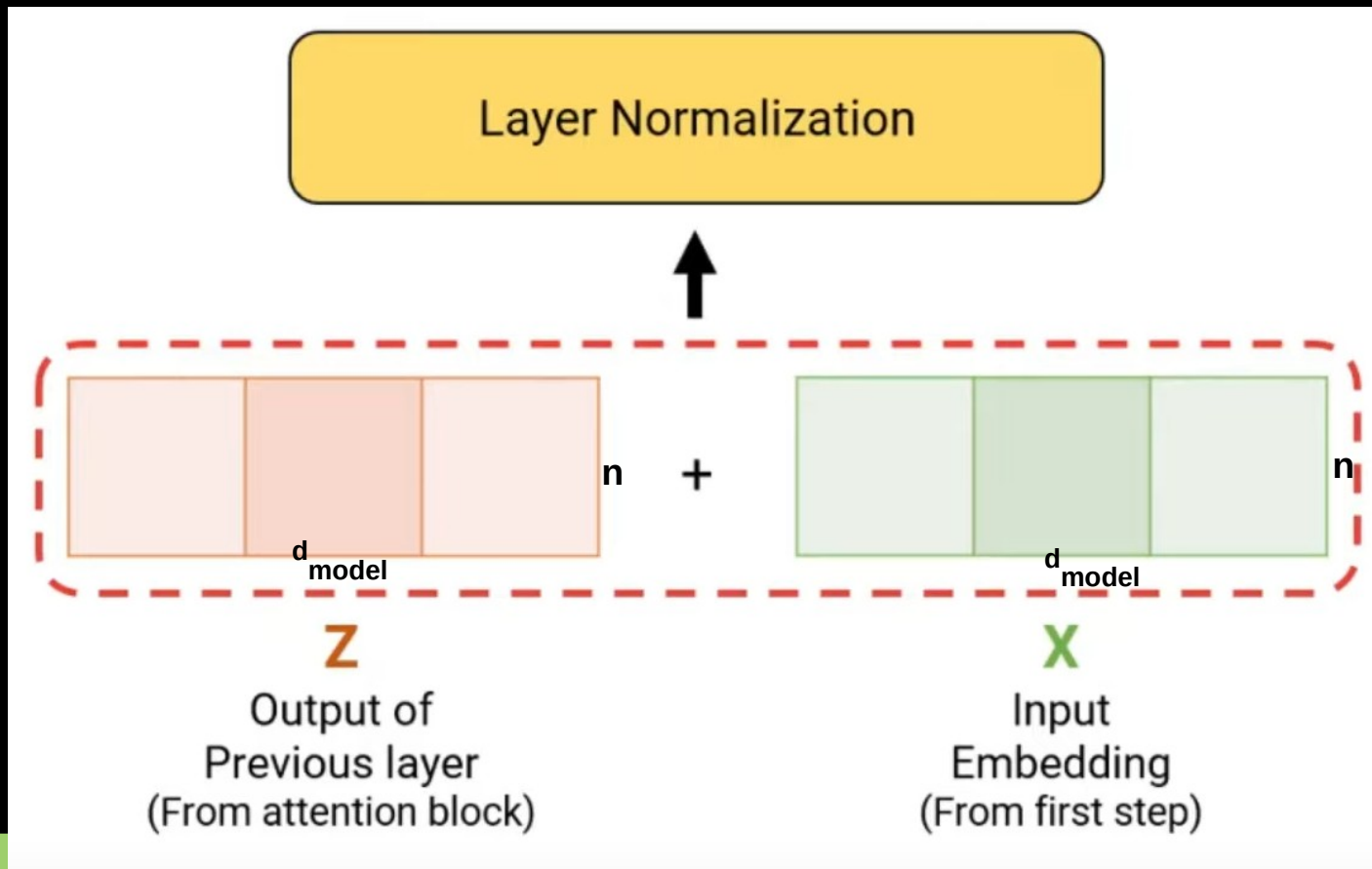
Encoder

1. Inputs embedding
2. Positional encodding
3. Multi-head self-attention
- 4. Add & Norm**
5. Feed forward
6. Add & Norm

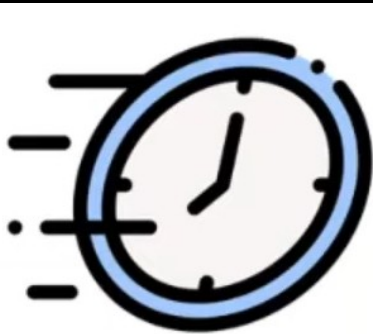
Transformer Architecture



Add (& Norm)



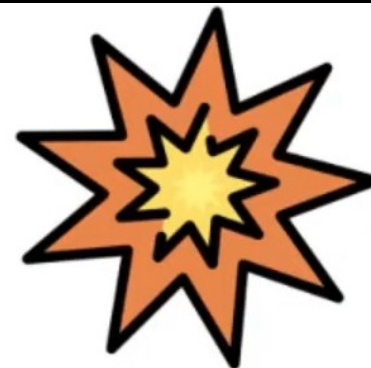
(Add &) Norm



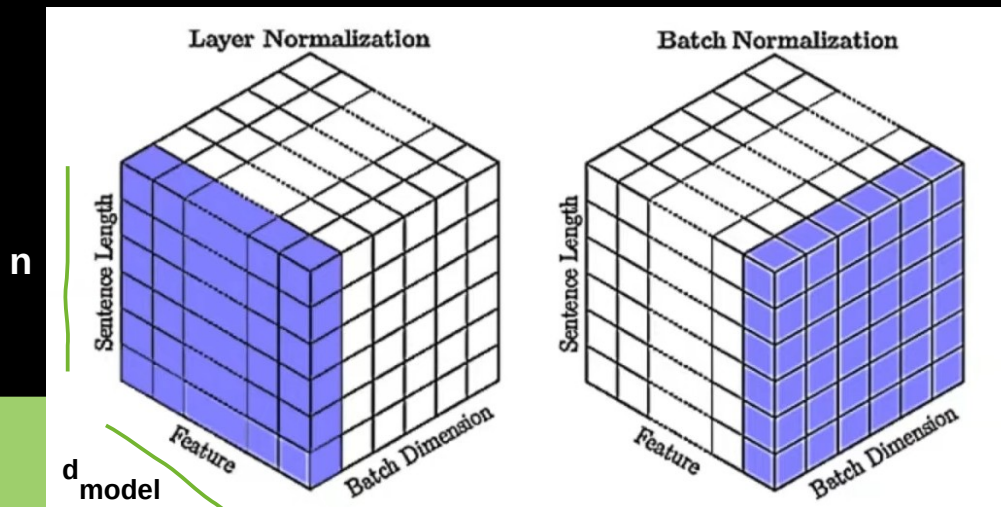
Faster Training



Reduce Bias



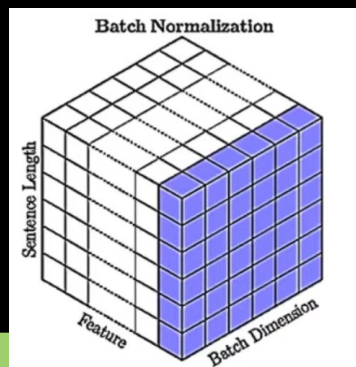
Prevent weight explosion



(Add & Norm) -> Batch normalization

Popcorn	0.31	0.14	0.93
Popped	0.14	0.88	0.98
Tea	0.85	0.20	0.14
Steeped	0.46	0.61	0.49

- Average = 0.44
- Variance = 0.07



Popcorn	-0.49	0.14	0.93
Popped	-1.14	0.88	0.98
Tea	1.56	0.20	0.14
Steeped	0.076	0.61	0.49

✓ **Normalized!**

- Average = 0
- Variance = 0.99

(Add & Norm) -> Layer normalization

Popcorn

0.31	0.14	0.93
0.14	0.88	0.98

Popped

- Average = 0.56
- Variance = 0.09

Tea

0.85	0.20	0.14
0.46	0.61	0.49

Steeped

- Average = 0.45
- Variance = 0.05



Popcorn

-0.83	-1.38	1.20
-1.38	1.04	1.37

Popped

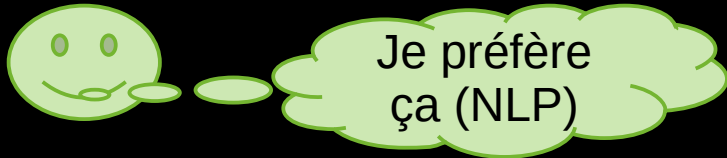
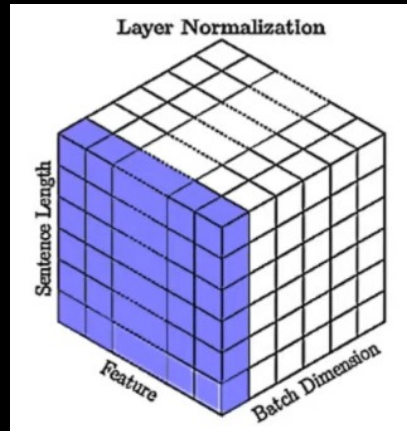
- ✓ **Normalized!**
- Average = 0
- Variance = 0.99

Tea

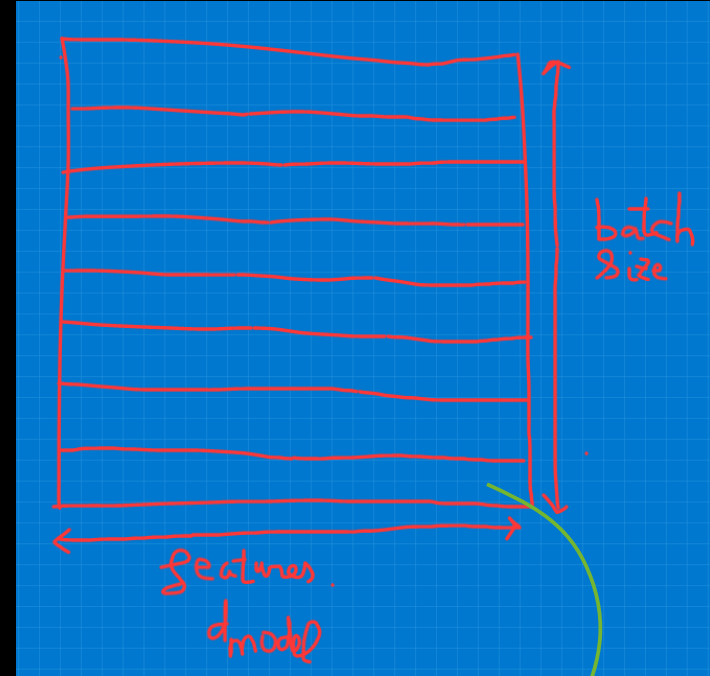
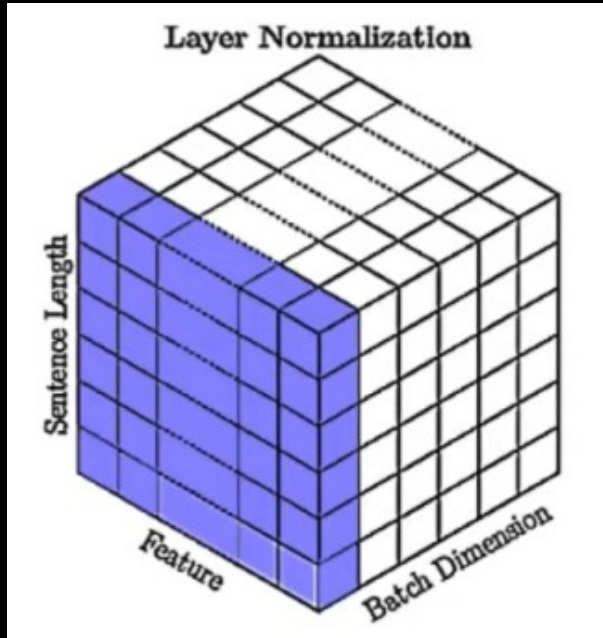
1.69	-1.11	-1.37
0.01	0.65	0.13

Steeped

- ✓ **Normalized!**
- Average = 0
- Variance = 0.99



(Add & Norm) -> Layer normalization

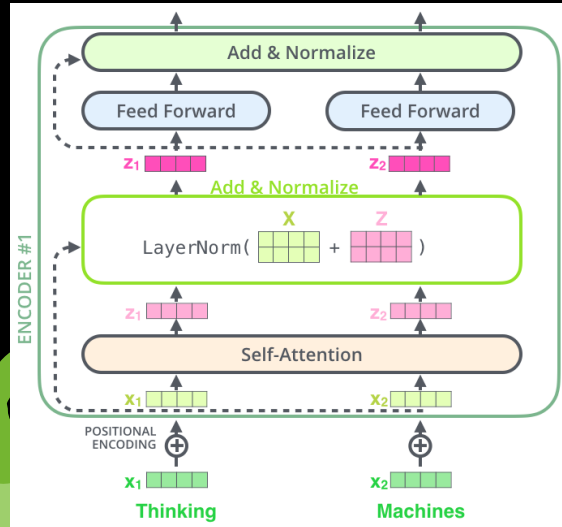


-- > une phrase (plusieurs z_i)

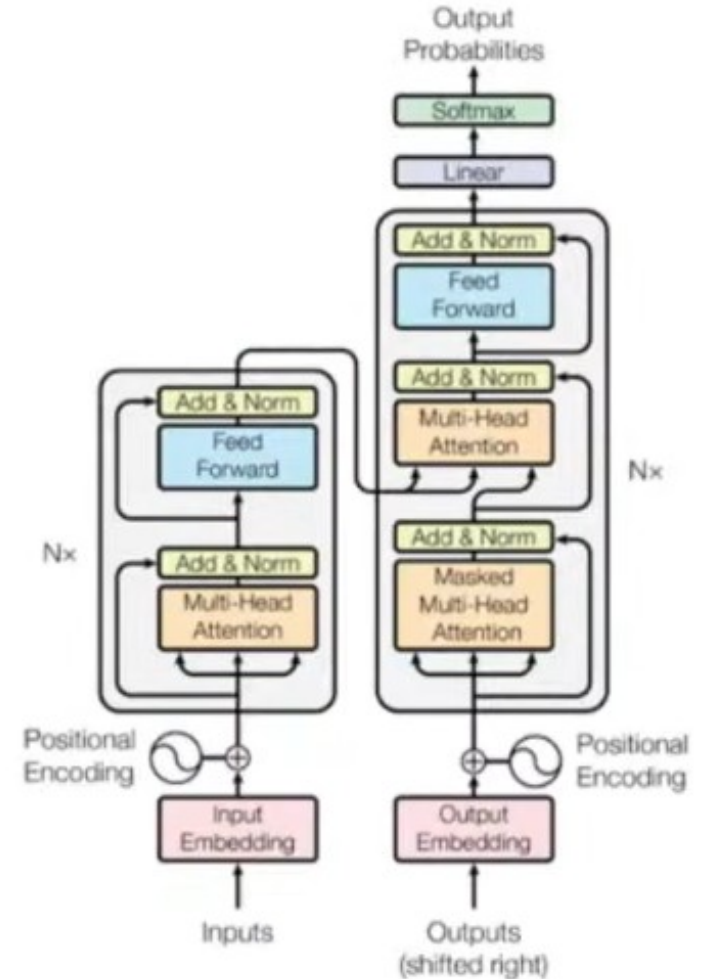


Encoder

1. Inputs embedding
2. Positional encodding
3. Multi-head self-attention
4. Add & Norm
- 5. Feed forward**
6. Add & Norm



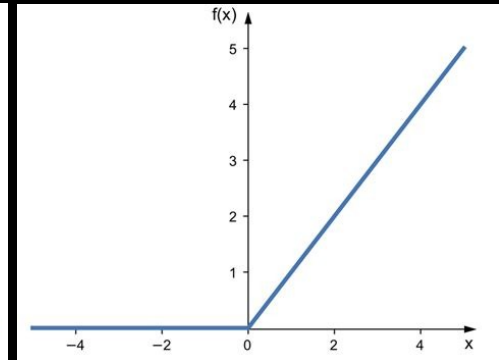
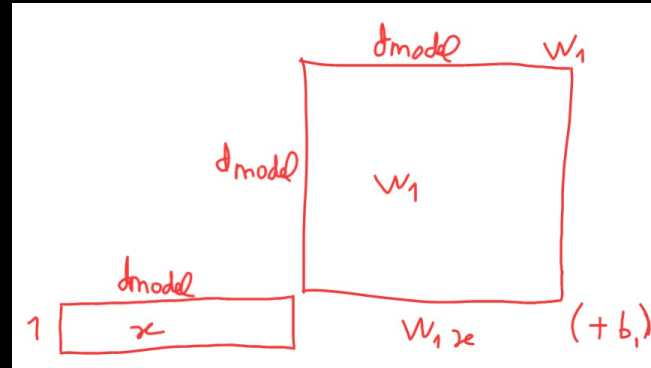
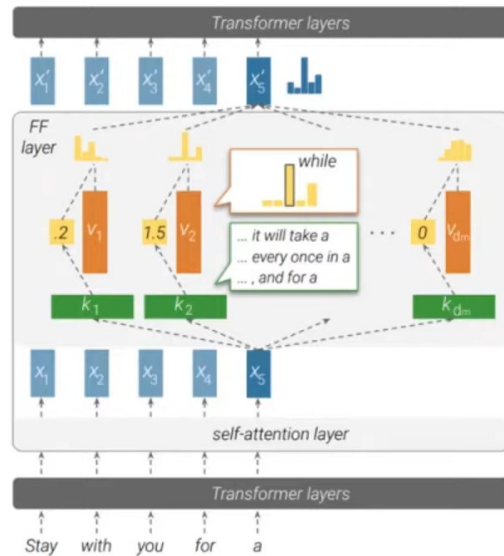
Transformer Architecture



Feed forward

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$



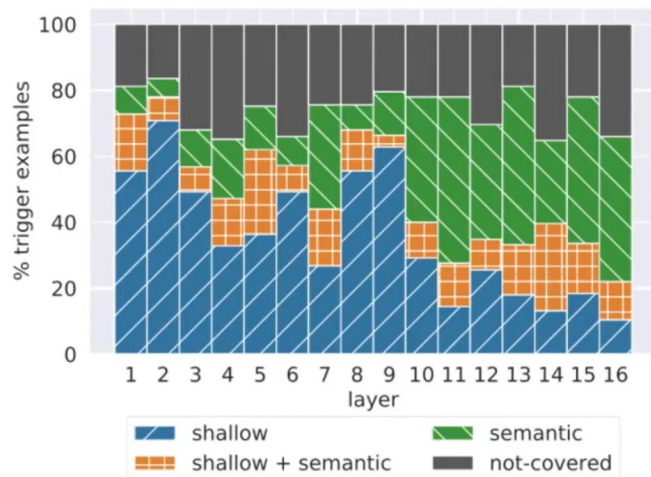
Objectif : mieux entrainer notre modèle pour utiliser d'autres outputs de self-attention.

ReLU permet plus d'expressivité /t sigmoid



Feed forward

What do Feed forward layers learn?



Geva, Mor, et al. "Transformer Feed-Forward Layers Are Key-Value Memories."

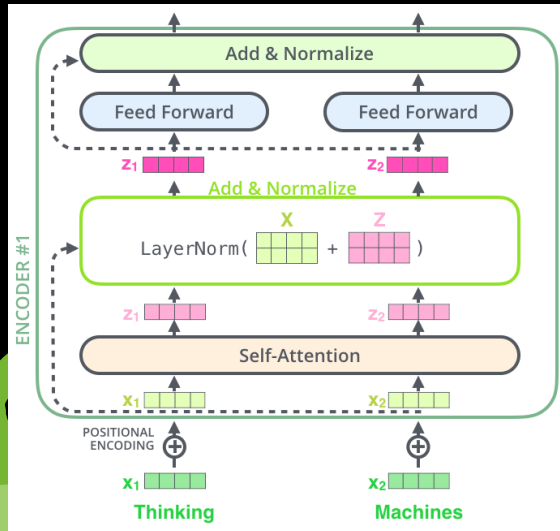
Shallow patterns	- At the meeting, Elton said that "for artistic reasons there could be no substitutes - In German service, they were used as substitutes - Two weeks later, he came off the substitutes
Shallow + Semantic Patterns	- On 1 April the <u>SRSG</u> authorised the <u>SADF</u> to leave their bases <u>Aircraft</u> from all four <u>carriers</u> attacked the Australian base <u>Bombers</u> flying <u>missions</u> to Rabaul and other Japanese bases
Semantic Patterns	- Time shifting viewing added 57 percent to the <u>episode's</u> - The first <u>season</u> set that the <u>episode</u> was included in was as part of the - From the original <u>NBC</u> daytime version , archived



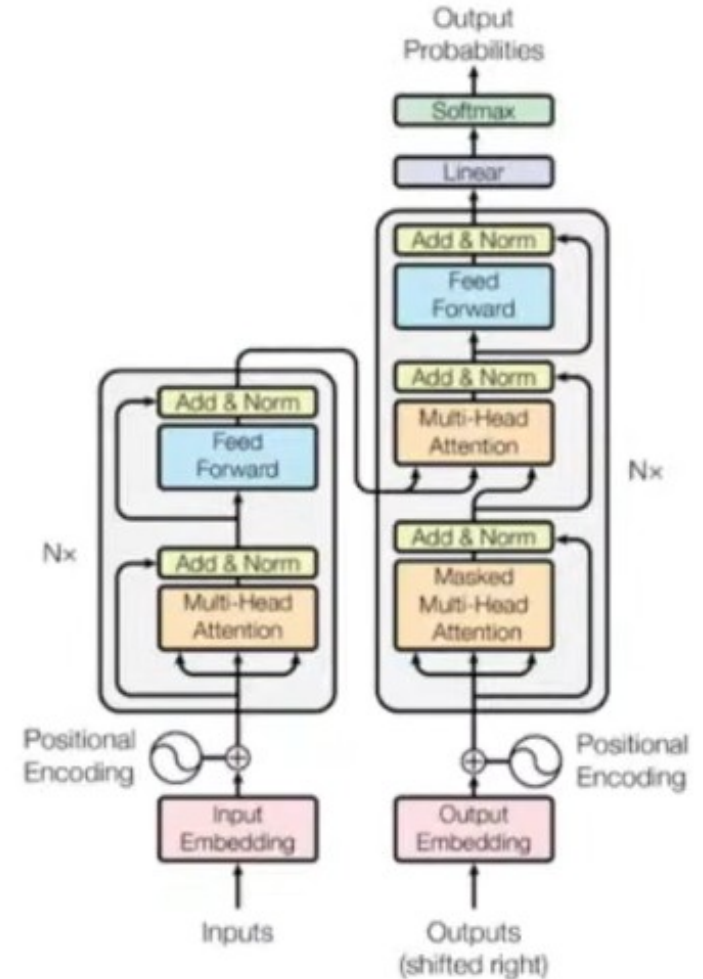
Encoder

1. Inputs embedding
2. Positional encodding
3. Multi-head self-attention
4. Add & Norm
5. Feed forward
6. Add & Norm

On connaît déjà...

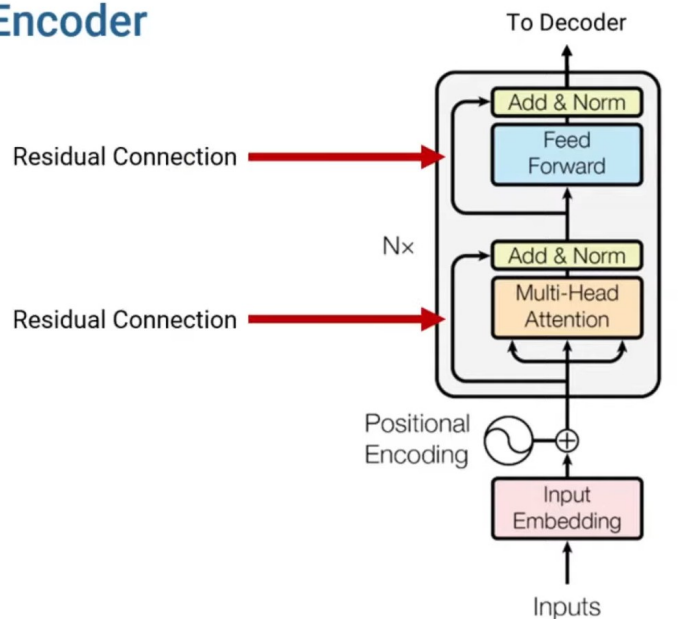


Transformer Architecture



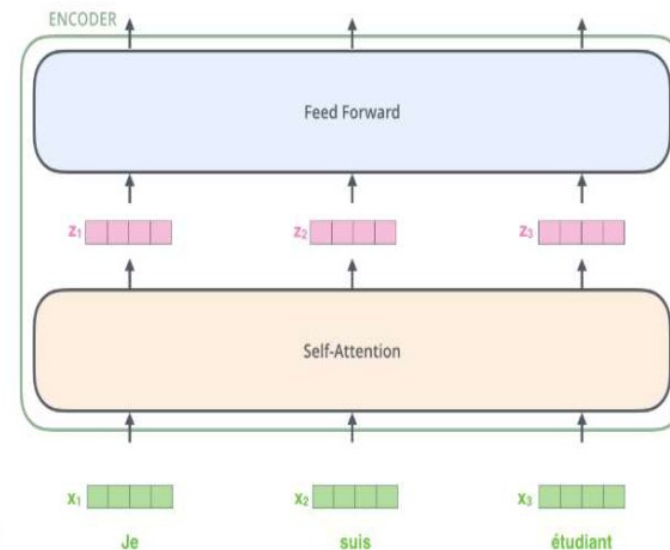
Rappel

Encoder



Récapitulation: L'encodeur dans le Transformateur

- Les entrées de l'encodeur passent d'abord par une **couche d'auto-attention** qui l'aide à regarder les autres mots de la phrase d'entrée lorsqu'il encode un mot spécifique
- Les sorties de la couche d'auto-attention sont acheminées vers un réseau de neurones à propagation avant FFNN
- Le même réseau FFNN est appliqué indépendamment à chaque position.
- Le mot dans chaque position traverse son propre chemin dans l'encodeur



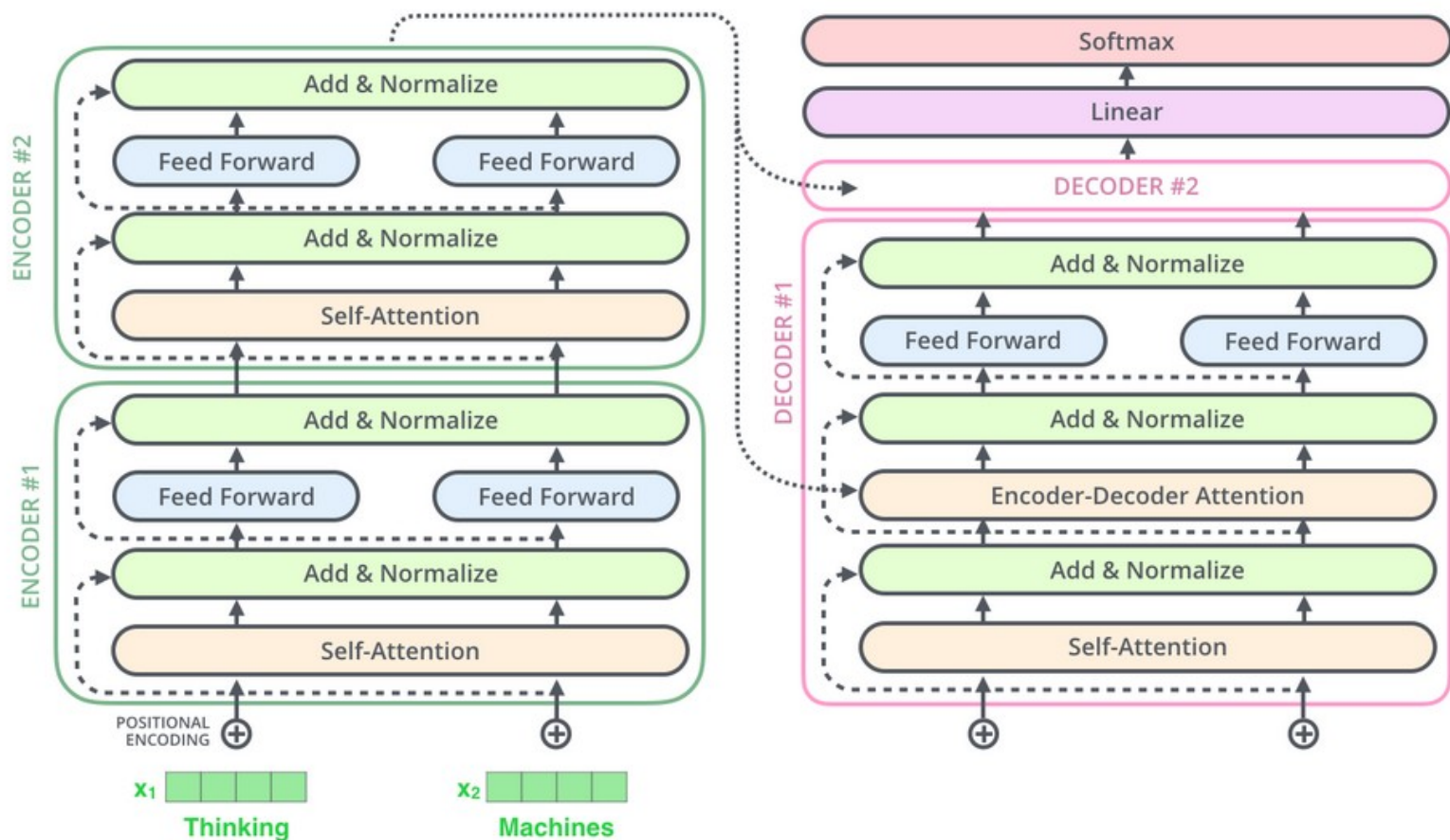
Source: <http://jalammr.github.io/illustrated-transformer/>

Prof. Amal Zouaq - Polytechnique Montréal (A 2023)

Détails du modèle (Encoder &) **Decoder**



Structure globale



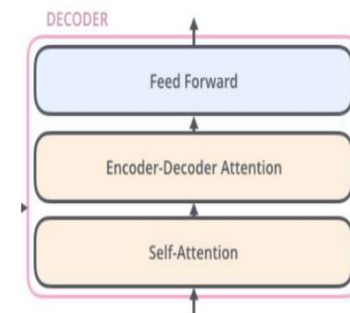
Structure globale

L'auto-attention dans le décodeur

- L'auto-attention sert ici à faire attention aux différents mots de la séquence en sortie
- La seule différence avec l'encodeur, est que la couche d'auto-attention ne peut voir que les positions précédentes de la séquence en sortie
- Cela est fait en masquant les positions suivantes (en leur attribuant la valeur $-\infty$) avant l'étape du Softmax dans le calcul de l'auto-attention

Le décodeur dans le transformateur

- En plus de l'auto-attention et de la couche de propagation avant (Feed forward), chaque décodeur a aussi une **attention encodeur-décodeur**



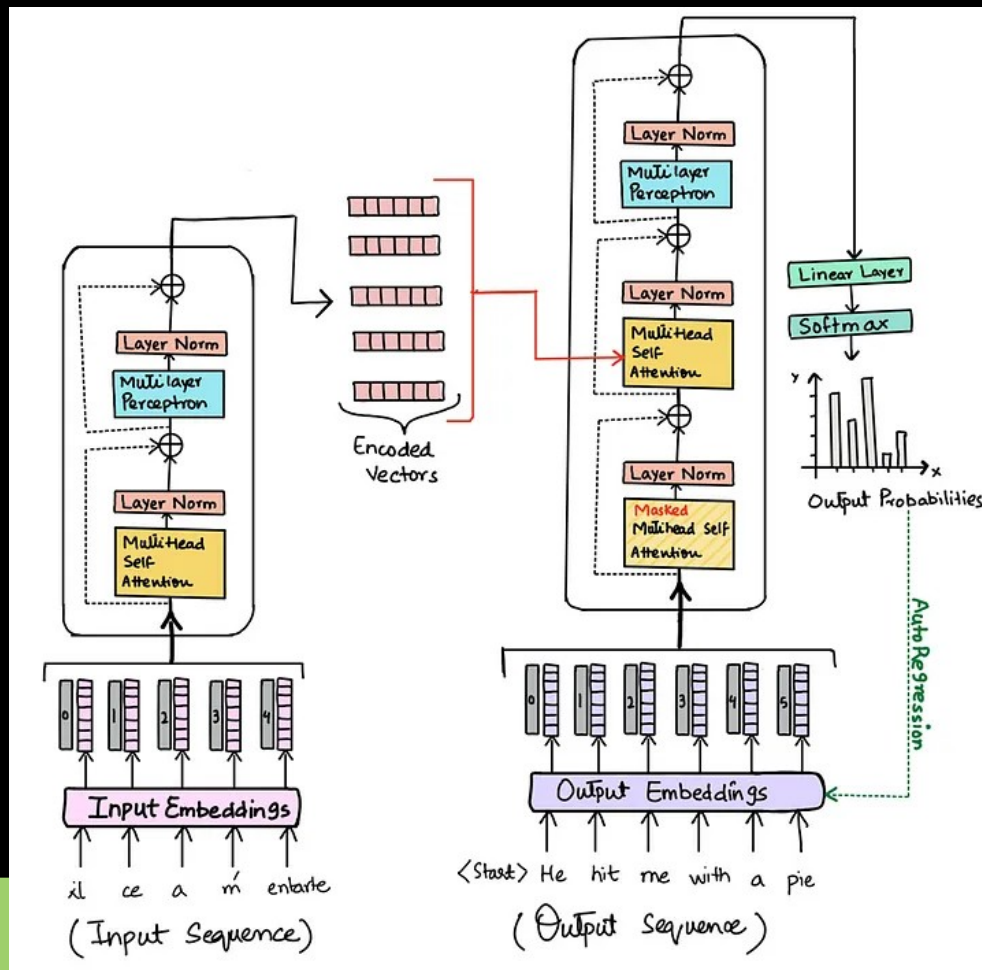
Source: <http://jalamar.github.io/illustrated-transformer/>

Attention encodeur-décodeur

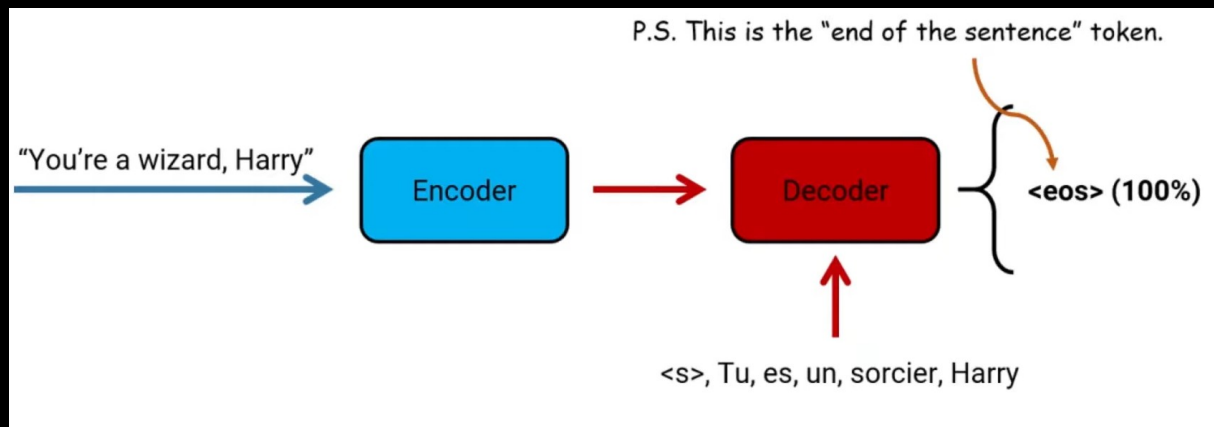
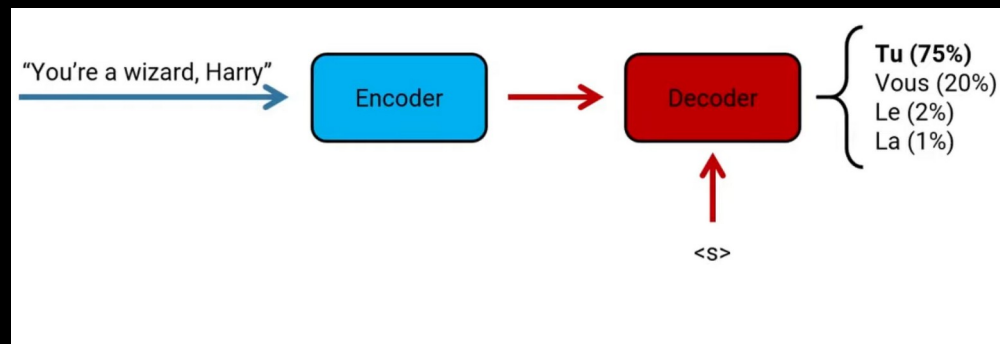
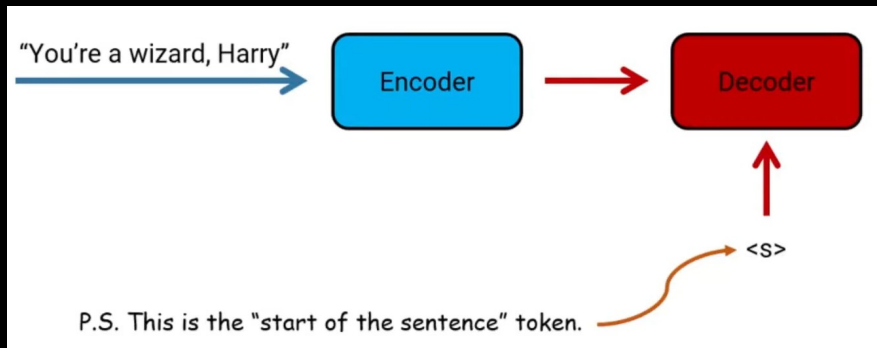
- L'**attention encodeur-décodeur** permet de déterminer les poids d'attention sur la sortie de l'encodeur qui permettent de générer le mot à la position courante du décodeur
- Le mécanisme d'attention reçoit les requêtes de la couche précédente du décodeur et les clés et les valeurs de la sortie de l'encodeur
- Dans un encodeur-décodeur:
 - La requête est représentée par l'état caché du décodeur
 - Les clés (*keys*) et les valeurs (*values*) sont les états cachés de l'encodeur



Structure globale



Application



Phase de training

Input sentence	Target sentence
"You're a wizard, Harry"	"Tu es un sorcier, Harry"
"We were on a break!"	"Nous avons rompu!"
"To infinity and beyond"	"vers l'infini et au-delà"



Phase de training : 1st self-attention layer

